

# Complete OSM Geocoder Implementation Manual

From PBF bytes to browser map, reverse geocoding, forward geocoding, POI search, and local Ollama intent parsing

Final implementation documentation

July 2026

This document explains the current implementation as a technical manual: simple enough to follow, but detailed enough to answer implementation questions about parsing, skipped data, memory layout, indexes, ranking, API endpoints, browser behavior, and local LLM handling.

## Contents

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>One-page mental picture</b>              | <b>6</b>  |
| <b>2</b> | <b>Input file: what an OSM PBF contains</b> | <b>6</b>  |
| 2.1      | PBF file . . . . .                          | 6         |
| 2.2      | Node drawing . . . . .                      | 6         |
| 2.3      | Way drawing . . . . .                       | 6         |
| 2.4      | Relation drawing . . . . .                  | 6         |
| 2.5      | Tags . . . . .                              | 6         |
| <b>3</b> | <b>Build and runtime shape</b>              | <b>8</b>  |
| 3.1      | Executable . . . . .                        | 8         |
| 3.2      | Main command modes . . . . .                | 8         |
| <b>4</b> | <b>Parser: what it keeps</b>                | <b>8</b>  |
| 4.1      | House records . . . . .                     | 8         |
| 4.2      | Street records . . . . .                    | 9         |
| 4.3      | Admin area records . . . . .                | 10        |
| 4.4      | POI records . . . . .                       | 10        |
| <b>5</b> | <b>Parser: what it skips</b>                | <b>11</b> |
| 5.1      | Skipped street-like ways . . . . .          | 11        |
| 5.2      | Unnamed street recovery . . . . .           | 11        |
| 5.3      | Skipped POIs . . . . .                      | 11        |
| 5.4      | Skipped raw data . . . . .                  | 11        |
| <b>6</b> | <b>Low-memory PBF reading</b>               | <b>12</b> |
| 6.1      | Why multiple scans exist . . . . .          | 12        |

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| 6.2       | The numbered PBF phases . . . . .                      | 12        |
| 6.3       | NeededNodeStore . . . . .                              | 13        |
| <b>7</b>  | <b>Administrative boundary assembly</b>                | <b>13</b> |
| 7.1       | Relation way spans . . . . .                           | 13        |
| 7.2       | Ring building . . . . .                                | 13        |
| 7.3       | Inner holes . . . . .                                  | 13        |
| <b>8</b>  | <b>Point-in-polygon and admin links</b>                | <b>14</b> |
| 8.1       | Point-in-polygon test . . . . .                        | 14        |
| 8.2       | Admin links for objects . . . . .                      | 14        |
| <b>9</b>  | <b>Street connection</b>                               | <b>15</b> |
| 9.1       | Why streets are connected . . . . .                    | 15        |
| 9.2       | How connection works . . . . .                         | 15        |
| <b>10</b> | <b>Compact dataset memory model</b>                    | <b>15</b> |
| 10.1      | String table . . . . .                                 | 15        |
| 10.2      | Geometry arrays . . . . .                              | 15        |
| 10.3      | Checked 32-bit offsets . . . . .                       | 16        |
| <b>11</b> | <b>Binary snapshot</b>                                 | <b>16</b> |
| 11.1      | What is written . . . . .                              | 16        |
| 11.2      | Why the forward index is inside the snapshot . . . . . | 16        |
| 11.3      | Snapshot version and layout checks . . . . .           | 16        |
| 11.4      | What still builds at server startup . . . . .          | 17        |
| <b>12</b> | <b>Server startup</b>                                  | <b>17</b> |
| 12.1      | Server constructor . . . . .                           | 17        |
| 12.2      | Spatial grid drawing . . . . .                         | 17        |
| <b>13</b> | <b>HTTP API</b>                                        | <b>18</b> |
| 13.1      | Stats endpoint . . . . .                               | 18        |
| 13.2      | Viewport endpoints . . . . .                           | 18        |
| 13.3      | Search endpoints . . . . .                             | 18        |
| 13.4      | Request validation and response boundaries . . . . .   | 18        |
| 13.5      | Measurement fields in deterministic search . . . . .   | 18        |
| <b>14</b> | <b>Frontend map</b>                                    | <b>19</b> |
| 14.1      | Layer display . . . . .                                | 19        |
| 14.2      | Viewport loading control . . . . .                     | 19        |
| 14.3      | Result highlighting . . . . .                          | 19        |
| 14.4      | Reference anchors in the map . . . . .                 | 20        |

|           |                                                               |           |
|-----------|---------------------------------------------------------------|-----------|
| 14.5      | LLM interpretation panel . . . . .                            | 20        |
| <b>15</b> | <b>Forward geocode index</b>                                  | <b>20</b> |
| 15.1      | Inverted index . . . . .                                      | 20        |
| 15.2      | Context index . . . . .                                       | 21        |
| 15.3      | Primary index . . . . .                                       | 21        |
| 15.4      | Substring suffix array . . . . .                              | 21        |
| 15.5      | Fuzzy search . . . . .                                        | 21        |
| 15.6      | How direct, substring, and fuzzy matching cooperate . . . . . | 22        |
| 15.7      | Conflict handling and match quality . . . . .                 | 22        |
| 15.8      | Expansion guards at Europe scale . . . . .                    | 23        |
| <b>16</b> | <b>Forward query matching</b>                                 | <b>23</b> |
| 16.1      | Normalization . . . . .                                       | 23        |
| 16.2      | Structured address planner . . . . .                          | 24        |
| 16.3      | Candidate intersection . . . . .                              | 25        |
| <b>17</b> | <b>Forward ranking formulas</b>                               | <b>25</b> |
| 17.1      | House ranking . . . . .                                       | 25        |
| 17.2      | Street ranking . . . . .                                      | 25        |
| 17.3      | Admin ranking . . . . .                                       | 25        |
| 17.4      | POI ranking . . . . .                                         | 26        |
| 17.5      | Additional candidate helpers . . . . .                        | 27        |
| 17.6      | Result-type filters, final sorting, and ties . . . . .        | 27        |
| 17.7      | What ranking can and cannot infer . . . . .                   | 28        |
| <b>18</b> | <b>Reverse geocoder</b>                                       | <b>28</b> |
| 18.1      | Input and output . . . . .                                    | 28        |
| 18.2      | Distance search . . . . .                                     | 28        |
| 18.3      | Reverse-result drawing . . . . .                              | 29        |
| <b>19</b> | <b>Natural-language and Ollama mode</b>                       | <b>29</b> |
| 19.1      | What Ollama is allowed to do . . . . .                        | 29        |
| 19.2      | Two verification passes . . . . .                             | 29        |
| 19.3      | Natural address guard . . . . .                               | 30        |
| 19.4      | Deterministic intent precedence . . . . .                     | 31        |
| 19.5      | Product and service concept graph . . . . .                   | 31        |
| 19.6      | Concrete addresses, named places, and vague anchors . . . . . | 31        |
| 19.7      | Reference-category search step by step . . . . .              | 32        |
| 19.8      | Local knowledge graph in depth . . . . .                      | 32        |
| <b>20</b> | <b>Frame of reference</b>                                     | <b>37</b> |

|           |                                                      |           |
|-----------|------------------------------------------------------|-----------|
| 20.1      | Map center . . . . .                                 | 37        |
| 20.2      | When place names override the current view . . . . . | 37        |
| 20.3      | When the current view helps . . . . .                | 37        |
| <b>21</b> | <b>Browser controls</b>                              | <b>37</b> |
| 21.1      | Geocoder text field . . . . .                        | 37        |
| 21.2      | All, Houses, Streets, Admin, and POIs . . . . .      | 38        |
| 21.3      | Start Ollama Service button . . . . .                | 38        |
| 21.4      | Reverse geocoder checkbox . . . . .                  | 38        |
| 21.5      | Viewport refresh and stale requests . . . . .        | 38        |
| <b>22</b> | <b>Performance controls</b>                          | <b>38</b> |
| 22.1      | Progress bars . . . . .                              | 38        |
| 22.2      | Why ETA can be unknown . . . . .                     | 39        |
| 22.3      | Why speed can drop during parsing . . . . .          | 39        |
| <b>23</b> | <b>Memory controls</b>                               | <b>39</b> |
| 23.1      | Released string interner index . . . . .             | 39        |
| 23.2      | Flat relation geometry . . . . .                     | 39        |
| 23.3      | Moved posting lists . . . . .                        | 39        |
| 23.4      | Packed geocode refs . . . . .                        | 40        |
| <b>24</b> | <b>How query shapes select a search path</b>         | <b>40</b> |
| <b>25</b> | <b>Complete implementation walkthrough</b>           | <b>40</b> |
| 25.1      | PBF primitives in the program . . . . .              | 40        |
| 25.2      | Tag reading and name selection . . . . .             | 41        |
| 25.3      | Address extraction from nodes . . . . .              | 41        |
| 25.4      | Address extraction from building ways . . . . .      | 42        |
| 25.5      | Address extraction from building relations . . . . . | 42        |
| 25.6      | Street extraction and street skip rules . . . . .    | 42        |
| 25.7      | Unnamed street recovery . . . . .                    | 43        |
| 25.8      | Street connection in full detail . . . . .           | 43        |
| 25.9      | POI extraction step by step . . . . .                | 44        |
| 25.10     | Administrative boundary extraction . . . . .         | 44        |
| 25.11     | Inner holes in admin areas . . . . .                 | 44        |
| 25.12     | Admin containment and object links . . . . .         | 45        |
| 25.13     | Reverse geocoder step by step . . . . .              | 45        |
| 25.14     | Forward geocode index construction . . . . .         | 46        |
| 25.15     | Forward search query path . . . . .                  | 46        |
| 25.16     | House ranking details . . . . .                      | 47        |
| 25.17     | Street ranking details . . . . .                     | 47        |

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| 25.18     | Admin ranking details . . . . .                   | 47        |
| 25.19     | POI ranking details . . . . .                     | 48        |
| 25.20     | Natural-language search path . . . . .            | 48        |
| 25.21     | Natural address guard . . . . .                   | 48        |
| 25.22     | Product and service concept handling . . . . .    | 49        |
| 25.23     | Browser map behavior . . . . .                    | 49        |
| 25.24     | Viewport loading . . . . .                        | 49        |
| 25.25     | Binary snapshot in detail . . . . .               | 50        |
| 25.26     | Server startup after loading a snapshot . . . . . | 50        |
| 25.27     | Progress bars, timings, and metrics . . . . .     | 50        |
| 25.28     | Memory reduction rules . . . . .                  | 50        |
| <b>26</b> | <b>Commands</b>                                   | <b>51</b> |
| 26.1      | Build . . . . .                                   | 51        |
| 26.2      | Parse Europe . . . . .                            | 51        |
| 26.3      | Load and serve . . . . .                          | 51        |
| 26.4      | Open browser . . . . .                            | 51        |
| <b>27</b> | <b>Closing mental model</b>                       | <b>51</b> |

# 1 One-page mental picture

The program is a local map engine. It does not call Google, Nominatim, Overpass, or any internet geocoding service. It reads an OpenStreetMap PBF file, keeps the objects that the local geocoder and map viewer actually use, writes a binary snapshot, loads that snapshot, and serves a browser map.

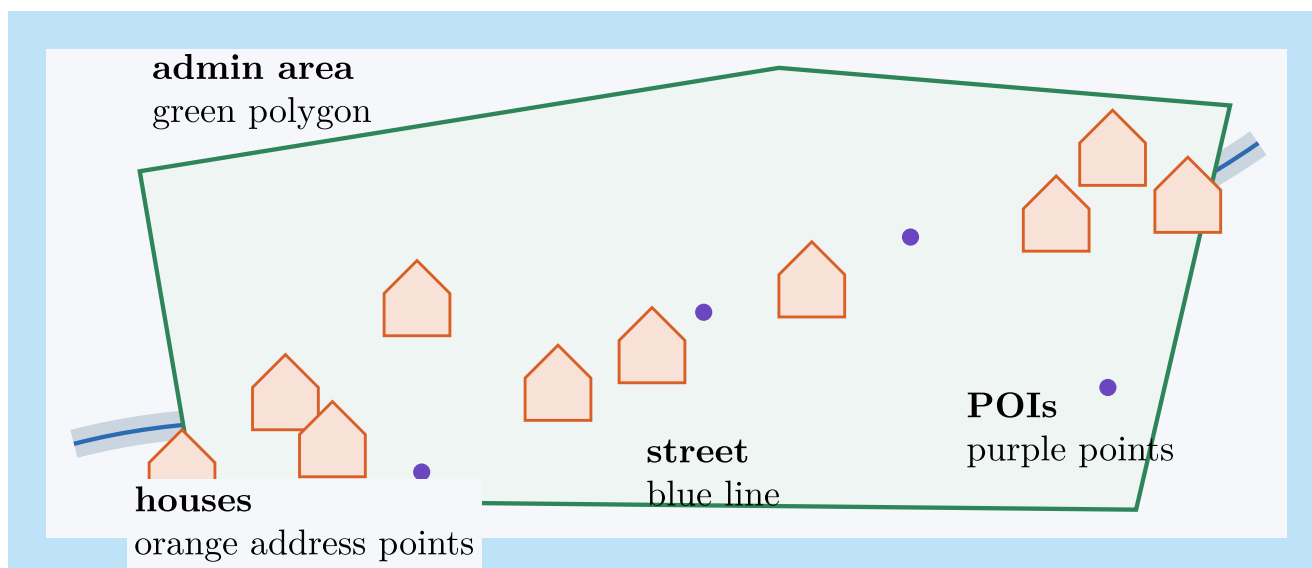


Figure 1: The reduced dataset shown as the program sees it: address points, street lines, admin polygons, and searchable POIs.

## 2 Input file: what an OSM PBF contains

### 2.1 PBF file

`.osm.pbf` is a compressed binary OpenStreetMap file. It is not an image. It stores billions of structured objects. The implementation uses **libosmium** to read this file.

The program reads three OSM primitive types:

| Primitive | What it is                                      | How this program can use it                                   |
|-----------|-------------------------------------------------|---------------------------------------------------------------|
| Node      | One coordinate with tags                        | house point, POI point, coordinate needed by a way            |
| Way       | Ordered list of node IDs plus tags              | street line, building address, POI area, admin boundary piece |
| Relation  | Group of nodes/ways/relations with member roles | administrative multipolygon, building relation address        |

### 2.2 Node drawing

### 2.3 Way drawing

### 2.4 Relation drawing

### 2.5 Tags

Tags are key-value pairs. The parser does not understand every possible tag. It keeps tags that are needed for houses, streets, admin areas, POIs, display names, ranking, and metrics.

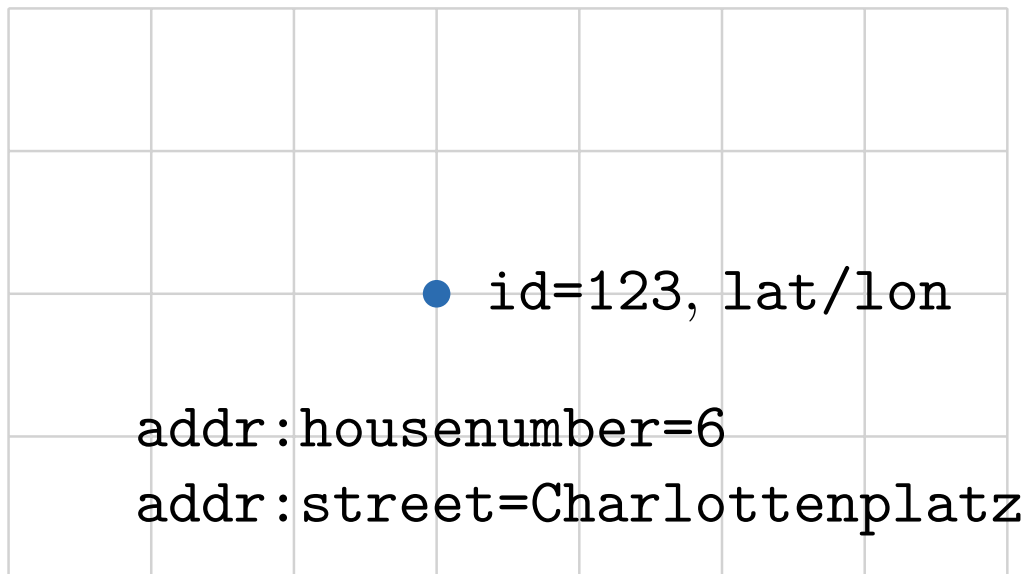


Figure 2: A node is one point. If it has address tags, the parser can store it as a house record.

highway=residential  
name=Bahnhofstrasse

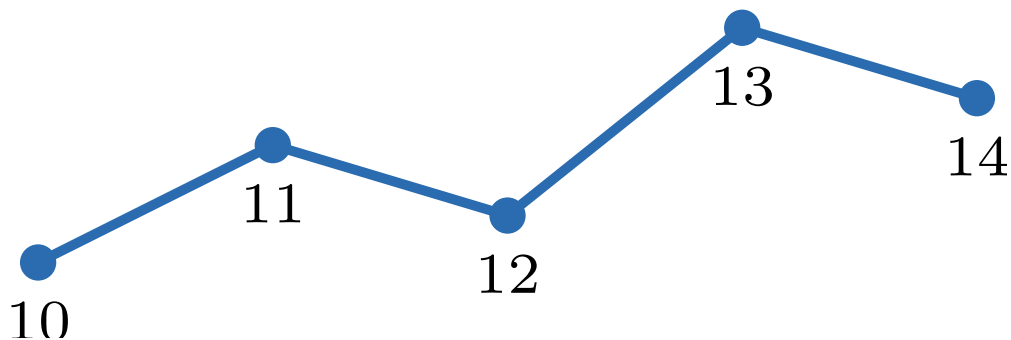


Figure 3: A way stores node IDs. The parser must recover the coordinates before it can draw a street line.

| Tag group                                    | Why it matters                                                                     |
|----------------------------------------------|------------------------------------------------------------------------------------|
| addr:housenumber,<br>addr:street             | Makes an OSM object useful as an address/house record.                             |
| addr:postcode,    addr:city,<br>addr:country | Strengthens address lookup and display.                                            |
| building=*                                   | Lets addressed ways/relations become house records.                                |
| highway=*                                    | Lets ways become street records, unless the highway type is skipped.               |
| name=*,    official_name=*,<br>ref=*         | Names and road references for display and search.                                  |
| name:de=*, name:en=*                         | Localized names when available. The selected parser language is German by default. |

|                                                          |                                                                                                                                                                             |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| boundary=administrative,<br>admin_level=*                | Creates admin areas used for hierarchy and ranking.                                                                                                                         |
| amenity=*, shop=*,<br>tourism=*, leisure=*,<br>landuse=* | Creates selected POIs.                                                                                                                                                      |
| brand=*, operator=*                                      | Brand is stored as its own searchable POI field. Operator is used as the display-name source only when name and brand are absent; PoiRecord has no separate operator field. |

---

## 3 Build and runtime shape

### 3.1 Executable

The executable is `osm_parser`. The build uses C++17, libosmium, protozero, zlib, bzip2, expat, cpp-httplib, and threads.

### 3.2 Main command modes

| Mode                               | Meaning                                                          |
|------------------------------------|------------------------------------------------------------------|
| <code>-parse input.osm.pbf</code>  | Read the PBF and build the compact dataset.                      |
| <code>-load-binary file.bin</code> | Load an already written compact dataset.                         |
| <code>-save-binary file.bin</code> | Save the compact dataset, including the forward geocoder index.  |
| <code>-server 8080</code>          | Start the local HTTP server and browser UI.                      |
| <code>-low-memory</code>           | Use extra sequential scans to reduce RAM pressure while parsing. |
| <code>-connect-streets</code>      | Merge touching street segments with the same identity.           |
| <code>-pbf-threads n</code>        | Configure libosmium PBF decoding threads.                        |
| <code>-geojson out.geojson</code>  | Export a bounded debugging GeoJSON sample.                       |

---

## 4 Parser: what it keeps

### 4.1 House records

A **house** in this program means an address object, not only a detached house. The parser keeps an OSM object as a house when it has `addr:housenumber` or `addr:street`. For ways and relations, it also requires `building=*`.

Stored fields:

- OSM ID
- representative coordinate
- house number
- street name
- postcode
- city
- country



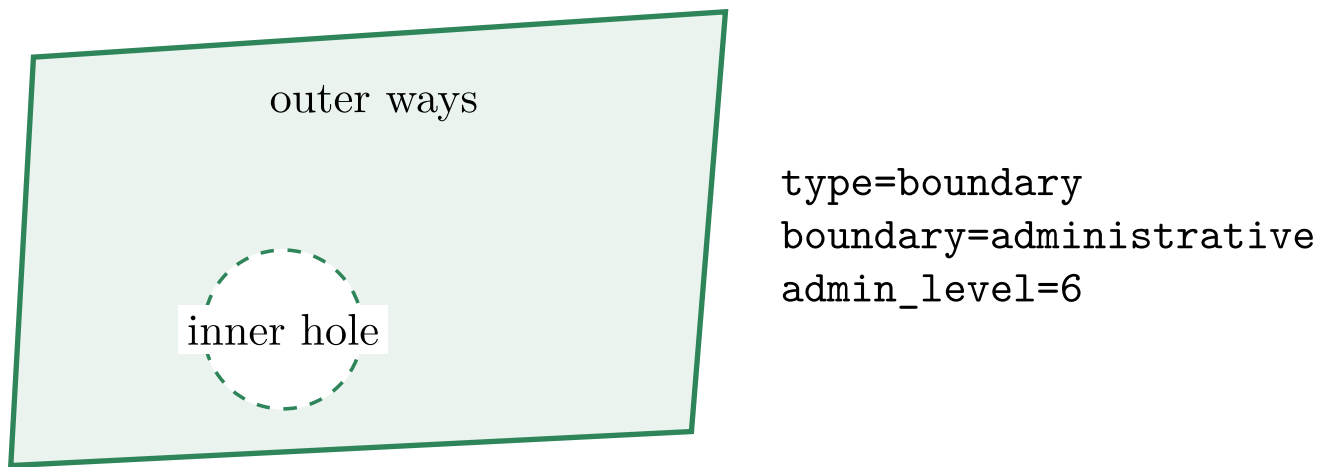


Figure 4: An administrative relation can have outer rings and inner rings. The implementation stores both.

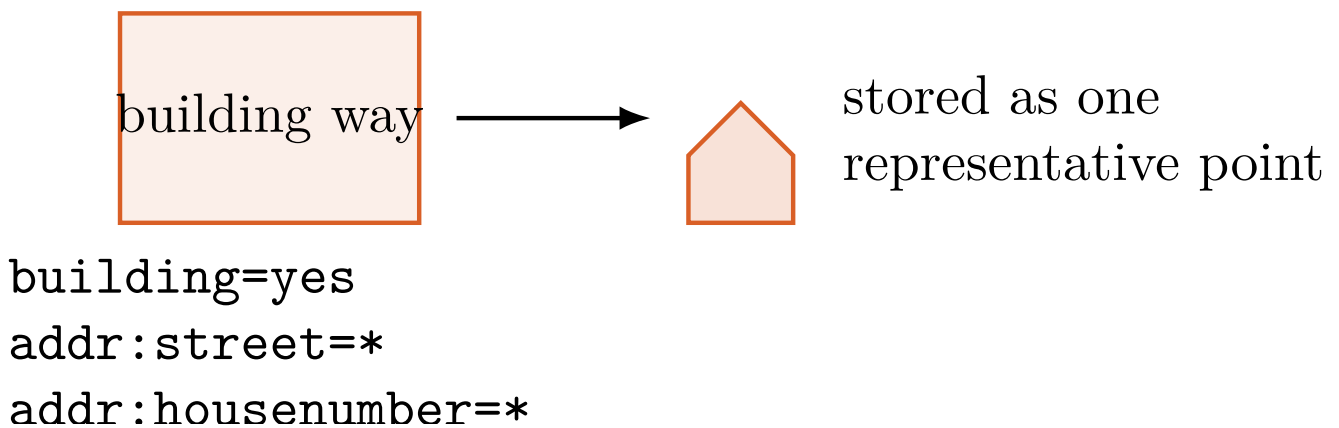


Figure 5: A building polygon with address tags is reduced to one address point.

- admin area link range
- source type: node, way, or relation

The parser keeps houses even when the street or house number is missing. Those incomplete addresses are weaker for exact forward geocoding, but useful for reverse geocoding and for measuring OSM data completeness.

## 4.2 Street records

A street record comes from an OSM way with `highway=*`, unless it is blocked or area-like.

The parser keeps:

- OSM ID
- street display name
- highway type
- geometry offset and size
- bounding box
- admin area link range

Street names are chosen with this order:

1. localized `name:<language>` when present

2. `name`
3. `localized_official_name:<language>` when present
4. `official_name`
5. `ref`, such as A 8, B 14, or L 558

### 4.3 Admin area records

Admin areas come from:

- administrative boundary ways
- administrative boundary relations

They require `boundary=administrative` and a valid `admin_level` from 2 to 12.

Stored fields:

- OSM ID
- `name`
- `admin_level`
- geometry/ring offsets
- parent area link range
- bounding box
- source type

### 4.4 POI records

POI means point of interest. The parser stores selected real-world places, not every tagged object.

Stored POI groups:

- **amenity**: fast food, restaurant, cafe, bar, pub, fuel, hospital, school, university, pharmacy, bank, parking
- **shop**: any non-empty shop except `shop=no`
- **tourism**: hotel, museum, attraction
- **leisure**: park, garden, playground, sports centre, swimming pool
- **landuse**: recreation ground, village green, stored as park-like

The POI display name is chosen from localized name, normal name, localized brand, brand, localized operator, operator.

The broad category and original tag value are deliberately separate. Every accepted `shop=*` record has broad category `shop`, but its tag value can remain `supermarket`, `chemist`, `bicycle`, or another real OSM shop value. Similarly, `leisure=garden` receives broad category `park` while retaining `garden`; `landuse=recreation_ground` and `landuse=village_green` also receive broad category `park`. Natural concept matching uses the original key/value for precise graph weights, while general display and broad category queries can use the category.

A POI record stores its OSM ID, point coordinate, string references for name, brand, category, tag key, and tag value, source primitive type, and offset/size into the shared POI-admin link array. Node POIs use their node coordinate. Way POIs use a representative point computed from resolved way geometry rather than retaining a second full POI polygon.

## 5 Parser: what it skips

### 5.1 Skipped street-like ways

The implementation skips street ways that do not help the intended road/address layer.

| Skipped case                                                                                                   | Reason                                                            |
|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <code>area=yes</code> , <code>area=1</code> , <code>area=true</code>                                           | A plaza or road area is not a centerline street.                  |
| <code>highway=construction</code>                                                                              | Not a usable final road.                                          |
| <code>highway=proposed</code>                                                                                  | Future road, not current navigable data.                          |
| <code>path</code> , <code>footway</code> , <code>cycleway</code> , <code>bridleway</code> , <code>steps</code> | Not part of the selected street/address layer.                    |
| <code>platform</code> , <code>corridor</code>                                                                  | Indoor/transport/platform geometry, not normal street centerline. |
| Unnamed highway ways                                                                                           | Normally skipped unless they match the unnamed recovery rule.     |

### 5.2 Unnamed street recovery

Some useful connector roads have no name. The parser can recover selected unnamed ways when they are useful connectors:

- `motorway/trunk/primary/secondary/tertiary` link ways
- `unclassified`, `residential`, `living street`, `road`
- `roundabouts`

This keeps map connectivity better without accepting every footpath or cycle path.

### 5.3 Skipped POIs

The parser skips POIs outside the selected tag rules. For example, random tags such as `bench=yes`, `surface=asphalt`, or `lit=yes` are not stored as searchable POIs. They are valid OSM data, but they are outside the program's search model.

### 5.4 Skipped raw data

The parser does not keep:

- raw node table for all Europe nodes
- all OSM tags
- complete OSM metadata such as `changeset/user/version/timestamp`
- relation members after assembly
- geometry for buildings after one representative address point is created
- every non-address building footprint

Those choices are deliberate: the server needs address, street, admin, POI, ranking, and display data, not a full OSM database clone.

## 6 Low-memory PBF reading

### 6.1 Why multiple scans exist

An OSM way contains node IDs, not coordinates. The simple method would load every node coordinate into a giant map. Europe has billions of nodes, so that would push RAM and swap too hard.

The implementation uses compact needed-node recovery:

1. scan ways and relations to learn which node IDs are needed
2. sort and bucket those IDs
3. scan nodes and fill only those coordinates
4. build the final ways/records using the compact node store

### 6.2 The numbered PBF phases

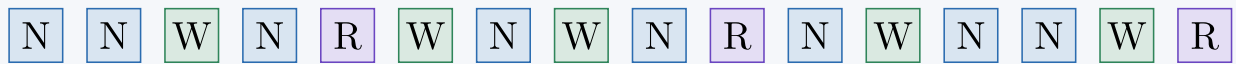
The progress label [n/9] in low-memory mode refers to a real sequence of full or entity-filtered PBF passes:

| Phase | Label                          | Purpose                                                                                                                                        |
|-------|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 1/9   | Administrative relation scan   | Reads relations only. Keeps administrative relation definitions, addressed building relations, member way IDs, roles, names, and admin levels. |
| 2/9   | Street endpoint recovery scan  | Reads ways and collects first/last node IDs from named, allowed street ways.                                                                   |
| 3/9   | Unnamed street recovery scan   | Reads ways again and keeps selected unnamed connector ways only when an endpoint touches the named-street endpoint set.                        |
| 4/9   | Address and boundary way scan  | Finds relevant non-street ways, relation member ways, addressed building ways, POI ways, and their required node references.                   |
| 5/9   | Address and boundary node scan | Stores coordinates only for the required IDs and extracts addressed/POI nodes.                                                                 |
| 6/9   | Address and boundary way build | Re-reads ways and builds final non-street records and flat relation member geometry from resolved coordinates.                                 |
| 7/9   | Street line way scan           | Collects node references needed by named streets and recovered unnamed connectors.                                                             |
| 8/9   | Street line node scan          | Resolves only those street node coordinates. It does not create house nodes again.                                                             |
| 9/9   | Street line way build          | Builds the final street geometry records.                                                                                                      |

Without `-low-memory`, phases 4–6 use one combined **Full dataset extraction** pass group, so the displayed PBF phase count is 6 rather than 9. The relation scan and two street-recovery scans still occur. Low-memory mode separates non-street and street node requirements so their temporary coordinate stores do not peak at the same time.

After the numbered PBF scans, the program performs in-memory work that is timed separately: building relation houses, assembling outer/inner admin rings, releasing temporary relation geometry, optionally connecting streets, assigning administrative links, building the forward text index, and writing the binary snapshot.

## PBF as a long shelf of objects



scan



The program walks the file sequentially.  
It does not keep the whole raw file in RAM.

Figure 6: Sequential scans are cheaper than holding every raw node coordinate in memory.

### 6.3 NeededNodeStore

`NeededNodeStore` stores only node IDs required for selected ways and relations. It uses:

- sorted node IDs
- coordinate array
- compact found-bit storage
- bucket starts for faster lookup

This is why `-low-memory` is recommended for Europe on a laptop. It adds scans but avoids memory blowups.

## 7 Administrative boundary assembly

### 7.1 Relation way spans

Admin relations reference member ways. The implementation stores relation member geometry in flat arrays:

- `RelationWaySpan`: way ID, offset, size
- `relationWayCoords`: all coordinates in one vector

This avoids one small heap allocation per member way.

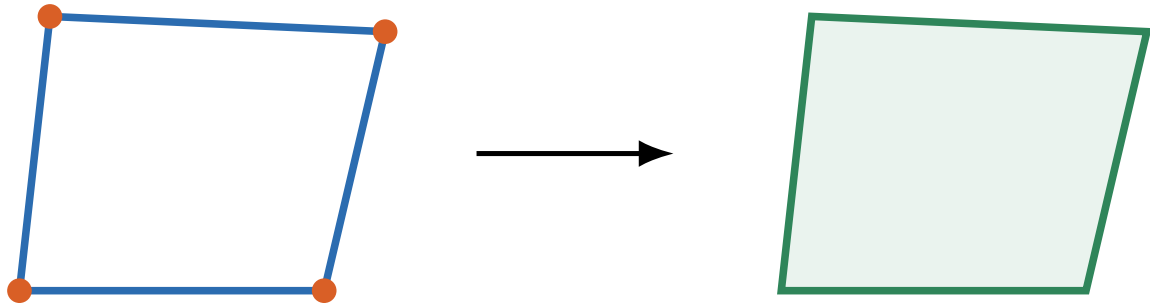
### 7.2 Ring building

Member ways are stitched into closed rings. A valid ring has at least 4 coordinates because the last coordinate closes the polygon.

### 7.3 Inner holes

Inner rings are assigned to the smallest outer ring that contains a probe point from the inner ring. At lookup time, a point is inside an admin area only if:

1. it is inside at least one outer ring
2. it is not inside any inner ring belonging to that admin area



separate member ways

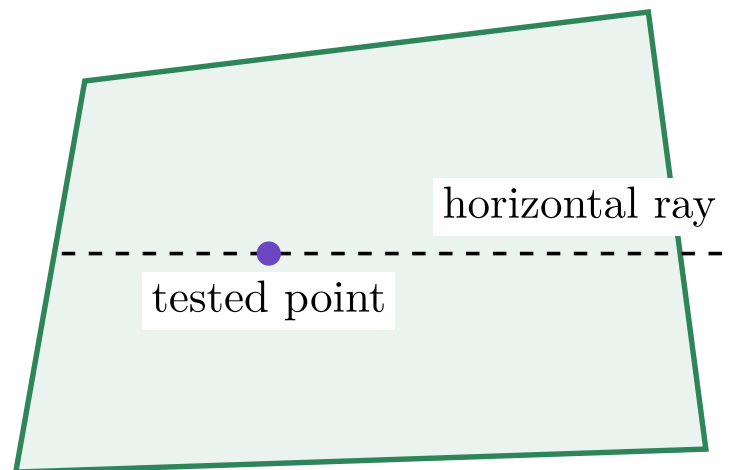
closed admin ring

Figure 7: Boundary member ways are assembled into polygon rings before point-in-polygon checks.

## 8 Point-in-polygon and admin links

### 8.1 Point-in-polygon test

The code uses a ray-crossing style polygon test. Instead of checking every polygon edge for every point, it also builds an edge index by latitude. A point only tests edges near that latitude.



If the ray crosses polygon edges an odd number of times, the point is inside. Inner rings subtract holes.

Figure 8: Admin containment depends on real geometry, not only bounding boxes.

### 8.2 Admin links for objects

The parser assigns:

- houses to containing admin areas
- street representative points to containing admin areas
- POIs to containing admin areas
- admin areas to parent admin areas

Bounding boxes and grid indexes prune candidates first. Polygon tests then confirm containment.

## 9 Street connection

### 9.1 Why streets are connected

OSM streets are often split into many ways. With `-connect-streets`, the program can merge touching street segments when they have the same name/highway identity.



Figure 9: Street connection reduces fragmented line records when endpoints touch.

### 9.2 How connection works

The implementation builds endpoint records, groups segments with equal identity, and uses a disjoint-set structure. Segments in the same connected component are merged into a connected street geometry. After connection, admin attribution is recomputed and the forward index is cleared so it can be rebuilt from the final street list.

## 10 Compact dataset memory model

### 10.1 String table

Repeated text is stored once in `data.strings`. Records store `StringRef`, a 32-bit number pointing into that table.

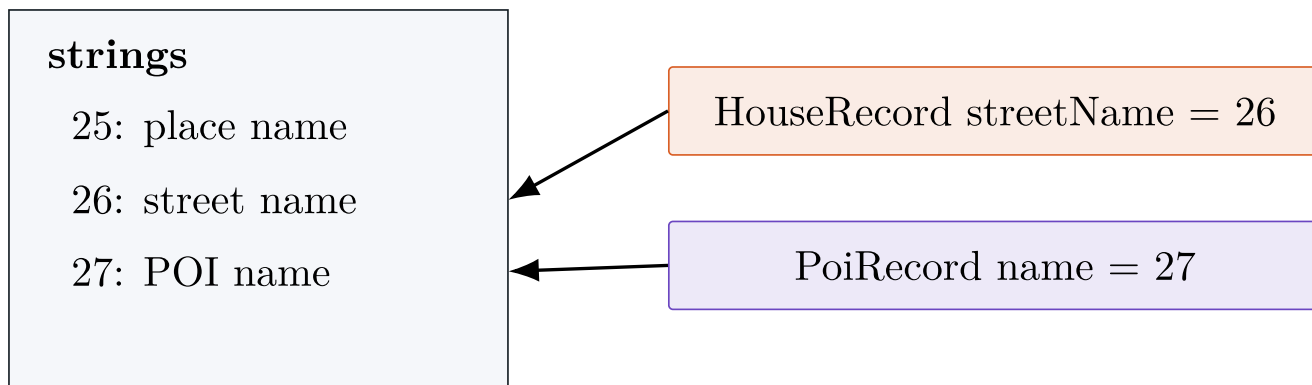


Figure 10: Records hold small references instead of full repeated strings.

### 10.2 Geometry arrays

Street and admin coordinates are stored in shared coordinate vectors. Records store offset and size.

- `streetGeometry`: all street line coordinates
- `adminGeometry`: all admin ring coordinates
- `geometryOffset`: where this feature starts
- `geometrySize`: how many coordinates it owns

## 10.3 Checked 32-bit offsets

Offsets and sizes are stored as `uint32_t`. The implementation uses checked casts when writing offsets, sizes, indexes, and link ranges. If a vector grows past the 32-bit range, the program throws an error instead of silently wrapping.

# 11 Binary snapshot

## 11.1 What is written

The binary snapshot stores:

- magic header
- selected name language
- string table
- houses
- streets
- admin areas
- POIs
- admin link arrays
- street geometry
- admin geometry
- admin rings
- statistics
- forward geocode index

## 11.2 Why the forward index is inside the snapshot

The forward index is expensive for Europe because it touches houses, streets, admin areas, and POIs. The final implementation writes it into the binary snapshot. Loading the binary no longer rebuilds the whole forward text index from scratch.

## 11.3 Snapshot version and layout checks

The dataset magic is the eight-byte value `OSMDB007`. The embedded forward-index block has its own eight-byte magic `FGIDX002`. Loading begins with a fresh empty `OSMDataset`; a wrong dataset magic, truncated vector, missing statistics block, wrong/missing forward-index magic, or unavailable forward index makes loading return failure.

Strings are written as length followed by bytes. Record, link, coordinate, ring, and suffix arrays are written as plain vector counts followed by their in-memory element bytes. Posting lists write each token string and its packed five-byte `GeocodeRef` vector. The snapshot is therefore a project-local binary format tied to the current record layout and version magic, not a portable OSM interchange format. The original PBF remains the source from which a new snapshot can be generated when the serialized data model changes.

Loading does not accept an old snapshot and silently rebuild missing text indexes. This is why a valid current snapshot starts search quickly after the runtime spatial grids are built, and why an incompatible snapshot is rejected instead of producing partially indexed results.



## 11.4 What still builds at server startup

The server still builds live spatial indexes:

- house grid
- street grid
- admin grid
- POI grid

These are server runtime structures used for viewport loading and reverse lookup.

## 12 Server startup

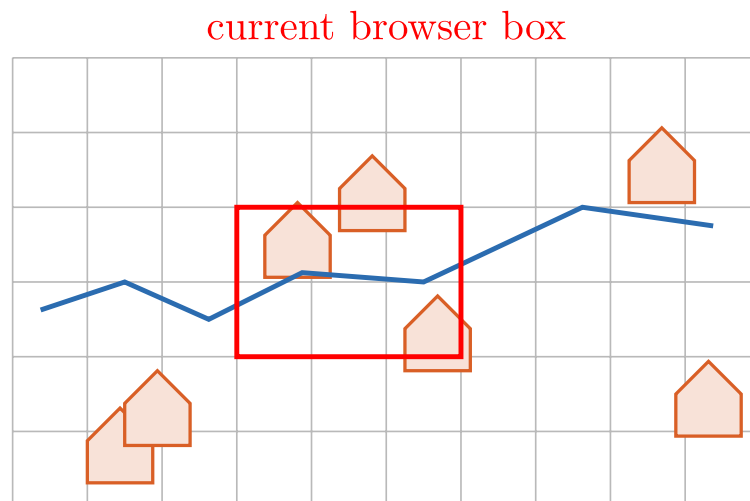
### 12.1 Server constructor

When the server starts, it receives a const reference to the loaded dataset. It does not copy the whole dataset. It builds runtime indexes that point into existing record vectors.

The house and POI grid uses 0.02-degree cells, the street grid uses 0.05-degree cells, and the admin grid uses 0.25-degree cells. Houses and POIs occupy the cell containing their point/bounding box. Streets and admin areas are inserted into every grid cell touched by their bounding box, so later geometry checks receive a candidate superset rather than missing a line or polygon that crosses a cell.

The constructor measures spatial-index build time separately. For the forward index, it only validates that the loaded snapshot contains an available index and stores a pointer to it; it does not duplicate or rebuild the posting lists. A snapshot without the required embedded index causes a clear runtime error.

### 12.2 Spatial grid drawing



Only cells touching the viewport are inspected for map loading.

Figure 11: The grid keeps browser requests from scanning the whole Europe dataset.

## 13 HTTP API

### 13.1 Stats endpoint

`/api/stats` returns counts, timings, memory estimates, and server index build metrics. The frontend uses it for the left statistics panel.

### 13.2 Viewport endpoints

- `/api/houses`: returns address points inside the map window
- `/api/streets`: returns street geometry inside the map window
- `/api/admin`: returns admin areas intersecting the map window
- `/api/admin-area`: returns one admin area geometry, optionally simplified

The frontend asks only for the current map box. This is why the browser can display Europe without downloading millions of objects at once.

### 13.3 Search endpoints

- `/api/geocode`: deterministic text search
- `/api/natural-geocode`: optional local LLM intent search
- `/api/reverse`: coordinate to nearest address/street/admin result
- `/api/ollama/start`: starts or restarts the configured local Ollama service and warms the configured model

### 13.4 Request validation and response boundaries

The server accepts both `q` and `query` for the two forward endpoints. A missing or empty query returns HTTP 400. The result limit is clamped to 1–100, with 20 as the default. Latitude/longitude must be numeric and inside valid geographic ranges; malformed reverse coordinates return HTTP 400 rather than entering spatial calculations.

The deterministic endpoint can also receive:

- `type=house|street|admin|poi`
- `lat` and `lon` as a ranking focus
- `minLat`, `maxLat`, `minLon`, and `maxLon` as viewport context

The natural endpoint receives the same map center and viewport from the browser when LLM mode is selected. These values are ranking context, not permission to fabricate or download results.

The HTTP layer installs error and exception handlers. An unknown route returns JSON with HTTP 404. An exception inside a route becomes a JSON server error instead of being allowed to unwind through the HTTP worker. The input checks and bounded search loops are part of the normal endpoint contract, not replacement results.

### 13.5 Measurement fields in deterministic search

The deterministic response exposes the normalized query, query time, primary/context index mode, active result-type filter, number of inspected posting references, anchor posting size, candidate count before ranking, and final result count. These fields make a slow or surprising query inspectable: a large

anchor posting means the token was broad, while a small candidate count after intersection means the other tokens pruned it strongly.

## 14 Frontend map

### 14.1 Layer display

The frontend is a single HTML page using Leaflet. It displays:

- orange house markers
- blue street polylines
- green admin polygons
- purple geocoder result markers
- reverse lookup popups

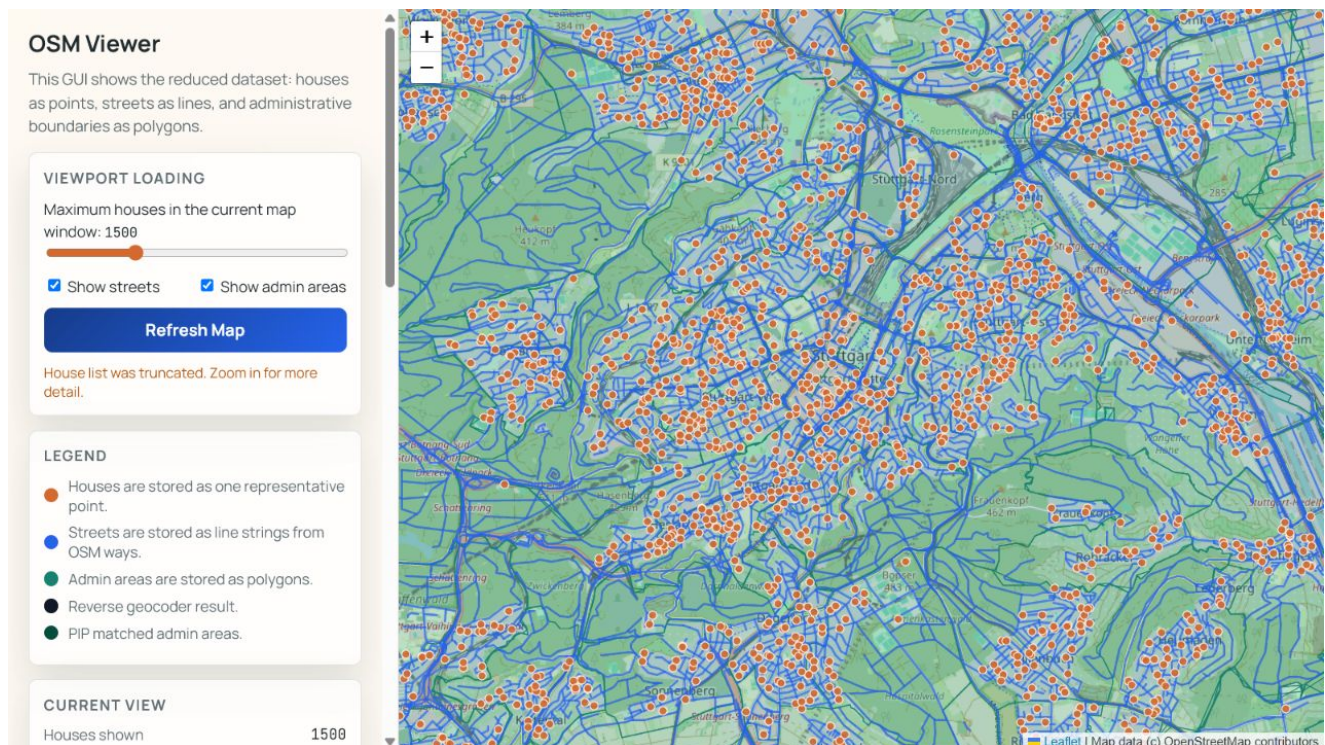


Figure 12: The running browser UI with houses, streets, admin polygons, controls, and statistics.

### 14.2 Viewport loading control

The slider limits how many house points are loaded in the current map window. If the map is zoomed out too far, the server can return a truncated set and the UI tells the user to zoom in for detail.

### 14.3 Result highlighting

Search results are drawn with highlighted markers. Admin result polygons can be fetched separately to avoid sending heavy geometry for every search result.

The default result list is mixed. The All, Houses, Streets, Admin, and POIs controls send the selected type to the server, which filters after scoring. Clicking one result changes only its presentation: point

markers grow and receive a yellow outline, while selected street geometry becomes a thicker yellow line. The click also moves the map to that result. Selection does not rerun or reorder the query.

Admin search results initially carry a representative point and compact metadata. The frontend requests the selected/result admin geometry separately from `/api/admin-area` and draws the polygon. This keeps the main geocoder JSON from repeating large boundary coordinate arrays for every result.

## 14.4 Reference anchors in the map

For a natural query that asks for one product or service near a generic reference category, the returned target POI remains the ranked result. When the backend uses a specific reference POI as the closest anchor, that result also contains a `referenceAnchor` object. The browser draws:

- the target POI as the normal purple result marker
- the reference POI as a smaller light-blue marker with a dark-blue border
- a blue dotted line between the target and that exact anchor

The line documents which anchor influenced the result; it is not a route and does not follow roads. Up to 48 reference anchors can be collected by the backend, up to 12 are used as search centers, and each final POI records only its closest chosen reference anchor.

## 14.5 LLM interpretation panel

When natural mode returns LLM diagnostics, the result panel displays:

- **LLM meaning**: the model's one-sentence interpretation
- **Ollama JSON**: the final JSON after the two verification passes
- **Backend JSON**: the structured intent actually accepted by C++
- **LLM checks**: the number of completed verification passes

This is a debugging view, not hidden chain-of-thought. The prompt explicitly asks for one short sentence describing the final intended local OSM search. The backend JSON is the authoritative description of what was executed; it may be stricter than the model's draft.

# 15 Forward geocode index

## 15.1 Inverted index

The forward geocoder uses an inverted index. A token points to records that contain that token.

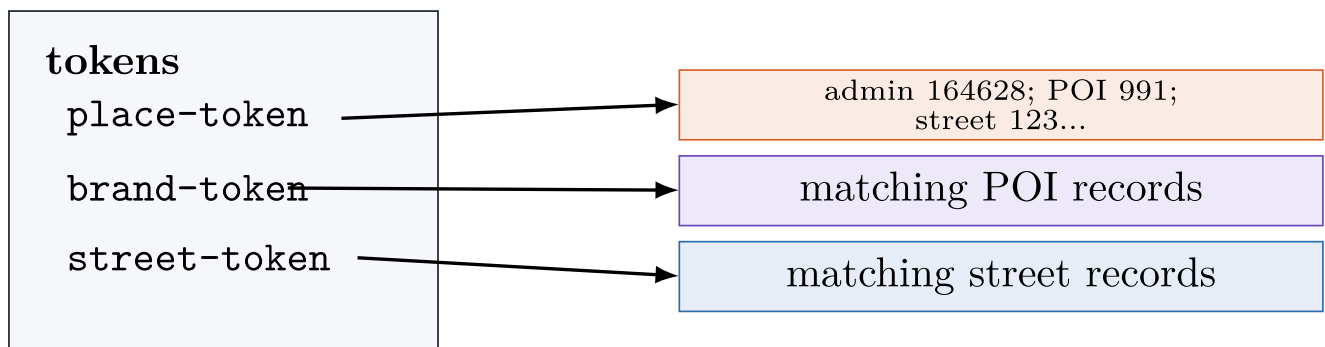


Figure 13: The geocoder starts from text tokens and jumps straight to candidate records.

## 15.2 Context index

The context index stores broad search context:

- house street, number, postcode, city, country, admin names
- street name and admin names
- POI name, brand, category, tag value, admin names
- admin area name and parent names

To keep context growth bounded, an object's searchable admin context includes at most three linked admin names. Context is required for numbered/postcode address lookup, where city or region words must lead to the house even though they are not house-owned fields.

## 15.3 Primary index

The primary index stores stronger object-owned fields:

- house street, number, postcode
- street name
- POI name, brand, category, tag value
- admin area name

Every query without a number or postcode starts from the primary index. This includes multi-token street/place and brand/place searches. Queries containing a number or postcode use the context index.

This prevents an ordinary place word from anchoring millions of objects merely because they lie inside that place. Explicit street-place and brand-place helpers add linked admin context in a controlled way after primary candidate generation.

## 15.4 Substring suffix array

For substring search, the program stores suffixes of vocabulary tokens with length at least 3. Each suffix entry stores a 32-bit vocabulary index and a 16-bit byte offset. For a token such as `streetname`, the array contains entries beginning at `streetname`, `treetname`, `reetname`, and every later byte position. The suffix entries are sorted by the suffix text, so a fragment can be found with binary search instead of scanning the complete vocabulary.

The query fragment must contain at least 3 normalized bytes. Matching vocabulary terms are ranked before they are returned to the query planner:

1. a vocabulary term that starts with the fragment is preferred over one that only contains it later
2. the smaller difference in token length is preferred
3. alphabetical order breaks the remaining tie

The substring routine examines at most 5,000 matching vocabulary terms in one call. The caller then requests only the number of expansions required by that query path. The main one-token path requests up to 96 substring expansions; later recovery paths request up to 64. These are candidate limits, not promises that every fragment has that many sensible meanings.

## 15.5 Fuzzy search

Fuzzy search corrects small insertion, deletion, or replacement mistakes with bounded Levenshtein distance:

- token length under 4: no fuzzy expansion
- token length 4 or 5: max edit distance 1
- longer token: max edit distance 2

The implementation does not scan the complete Europe vocabulary. It splits the misspelled token into overlapping seed substrings. The seed length is based on the token length and allowed edit distance, is never below 3, and is capped at 5. Each seed is looked up through the same suffix array used by substring search. This produces at most 20,000 vocabulary indexes per seed and at most 60,000 distinct vocabulary indexes over the complete fuzzy call.

Only those seed candidates are checked with bounded edit distance. A candidate is rejected immediately when its length differs by more than the allowed distance. The remaining candidates are ordered by:

1. smaller edit distance
2. longer common prefix with the typed token
3. shorter candidate length
4. alphabetical order

The main one-token path asks for at most 24 fuzzy expansions; later recovery paths ask for at most 12. Fuzzy search only returns vocabulary tokens already present in the local index. The distance implementation does not have a separate transposition operation. A swapped pair can still match when the resulting Levenshtein distance fits the length-dependent bound.

## 15.6 How direct, substring, and fuzzy matching cooperate

The three mechanisms are not three independent searches whose result lists are simply appended at the end. They cooperate while the posting list for each query token is being prepared.

1. The server first tries an exact normalized posting-list lookup.
2. It also prepares deterministic English/German place aliases and German missing-e variants where the code supports them.
3. For a one-token query, both substring and fuzzy expansions are considered even when the direct token exists. This is required for a short fragment that is also a real but less important word.
4. For a multi-token query, an existing direct posting is normally kept without expansion. Expansion is used when that token is missing, which prevents every ordinary address token from producing a large alternative set.
5. Posting lists for accepted alternatives are merged.
6. The merged references are sorted and deduplicated by the packed pair (`object type`, `record index`).
7. Multi-token queries then intersect the prepared lists, starting with the smallest one.

## 15.7 Conflict handling and match quality

Substring and fuzzy expansion can discover different vocabulary terms for the same input. A misspelled place token may reach a longer POI token through substring expansion while fuzzy verification reaches the intended admin-name token. This is expected candidate generation, not corruption.

For a one-token search, the server verifies the typed token and its deterministic alternatives against each candidate object's own tokens. The match-quality function uses the following score order:

| Token relationship                                                                  | Implemented treatment                                                                                          |
|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Exact normalized token                                                              | Base quality 1,400.                                                                                            |
| Street abbreviation such as <b>koenigstr</b> followed by <b>asse</b> or <b>asze</b> | Starts near 1,450 and is adjusted by common-prefix length and length difference.                               |
| Prefix match                                                                        | Starts at 1,150, then rewards common prefix and penalizes extra length.                                        |
| Contained substring of length at least 4                                            | Starts at 820, then applies prefix and length adjustments.                                                     |
| Fuzzy match                                                                         | Starts at 620, loses 170 per edit, rewards common prefix, and is rejected when the final quality is below 120. |
| No verified relationship                                                            | Quality 0; a one-token street, admin area, or POI candidate is discarded.                                      |

This gives exact and prefix relationships more influence than a fuzzy relationship. Object-level ranking still matters: an exact token on an obscure POI and a high-quality corrected token on an important admin area are scored using their own object-type formulas. The program therefore does not claim that one expansion source always wins without considering the object.

The expansion source is not stored as a permanent label on each final candidate. After references are merged, field verification and the normal ranker decide the order. This prevents the same OSM object from appearing twice merely because direct, substring, and fuzzy lookup all reached it.

## 15.8 Expansion guards at Europe scale

Large posting lists can represent extremely common words. To keep one typo from expanding into millions of references, the server ignores an expanded posting list above 30,000 references for a one-token query or above 250,000 references for a multi-token query. Direct postings are not discarded by this expanded-posting rule. These guards trade exhaustive recall for bounded latency and memory use; the response reports posting and candidate counts so the behavior can be measured.

## 16 Forward query matching

### 16.1 Normalization

Text is normalized before indexing and querying:

- ASCII letters become lowercase
- punctuation becomes spaces
- German umlauts become **ae**, **oe**, **ue**
- **ss** is used for the German sharp s (ß)
- UTF-8 combining marks in the handled combining-mark range are removed
- other unsupported non-ASCII bytes act as separators rather than receiving broad transliteration
- duplicate tokens are removed
- **str**, **strasse**, and related street spelling normalize toward **strasse**

Normalization is intentionally English/German-oriented. It is not a general Unicode normalization library and does not promise transliteration for every European alphabet. Original display strings remain unchanged in the string table; normalization only creates search tokens.

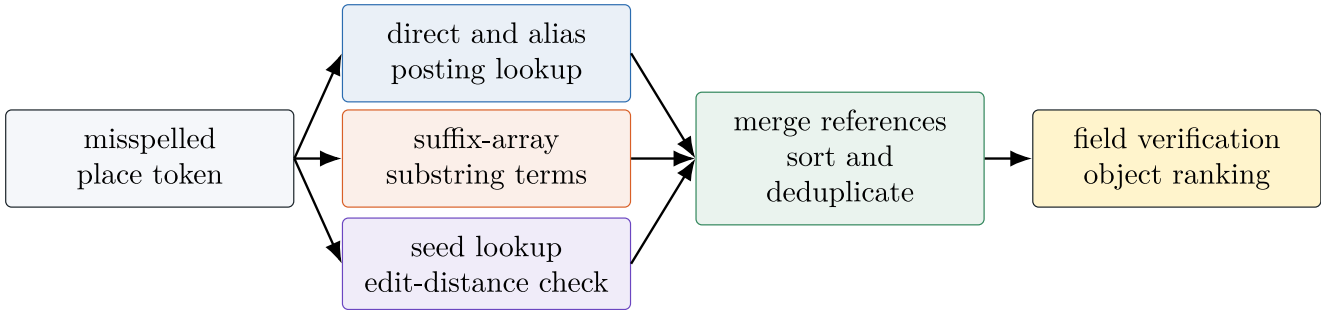


Figure 14: Substring and fuzzy expansion enlarge a token’s candidate posting list. Deduplication happens before object scoring.

`uniqueTokens` sorts tokens and removes empty/duplicate values. This is suitable for posting-list intersection, where token order does not matter. Separate ordered-token parsing is used by street/place and brand/place splitting because those helpers need to know which words appeared together on each side of a possible split.

## 16.2 Structured address planner

A numbered query is not searched as one inseparable phrase. The planner assigns each part a field before it accepts a house:

1. Separate postcode-shaped and house-number tokens from the ordered text tokens.
2. Test ordered text splits against exact loaded administrative names, including the implemented English/German place aliases.
3. Keep the recognized administrative side as the place restriction and the other side as the street phrase.
4. Find real street records whose own name tokens match the street phrase and whose linked local admin areas match the place restriction.
5. Keep a house only when its stored number equals the requested number, its stored street name equals one of those planned street names, and it shares the planned local admin context.

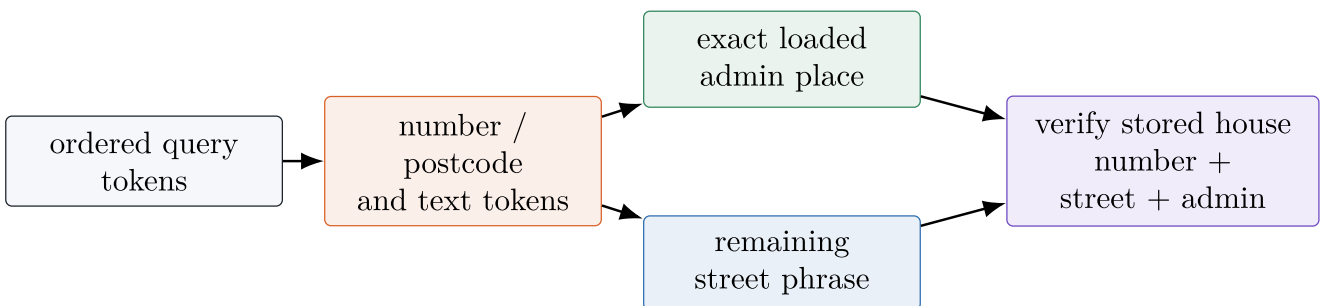


Figure 15: The address planner assigns fields first and accepts a house only after all stored address fields agree.

If a number is followed only by a recognized place name, no street phrase remains. The endpoint rejects that incomplete shape with `address query needs a street name`. It does not reinterpret the place as a street.

If a matching street exists but no exact stored house passes all three checks, the mixed view can return that street with `addressResolution=street_level`. The street-only view can also return it. The house-only view returns no result, because a street coordinate is not an address point. The planner never interpolates or invents a house coordinate.



## 16.3 Candidate intersection

The geocoder finds a posting list for each query token. It sorts posting lists by size and uses the smallest list as the anchor. A candidate must appear in every token posting list to pass the first stage.

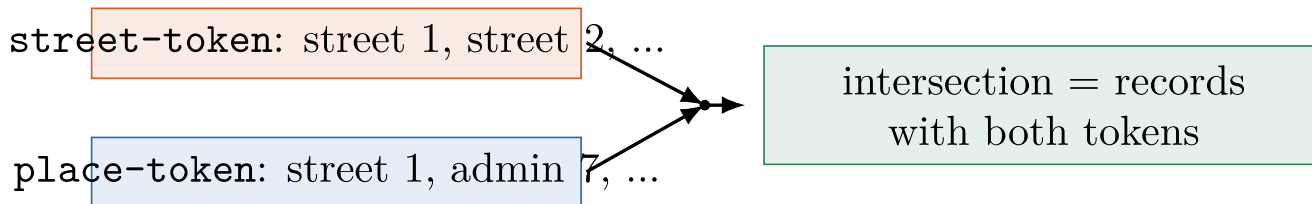


Figure 16: Candidates must match all query tokens before scoring.

## 17 Forward ranking formulas

### 17.1 House ranking

House candidates are considered only when the query has a number. The number must match. If a postcode is present, the postcode must match.

House score:

$$\begin{aligned} \text{score} = & 120 + 15 \cdot \text{matchedTokens} + 80 \cdot \text{numberMatch} \\ & + 45 \cdot \text{postcodePresent} + 25 \cdot \text{streetMatches} \\ & + 12 \cdot (\text{cityMatches} + \text{adminMatches}) \end{aligned}$$

This makes exact numbered addresses stronger than loose text matches.

### 17.2 Street ranking

Street score:

$$\begin{aligned} \text{score} = & \text{base} + 15 \cdot \text{matchedTokens} \\ & + 25 \cdot \text{streetMatches} + 10 \cdot \text{adminMatches} \end{aligned}$$

Here **base** is 35 for number queries and 95 otherwise.

Additional terms:

- exact street name: +50
- street phrase appears in full query: +620 plus +240 per street token
- one-token match quality: three times the direct/prefix/substring/fuzzy token quality
- street name looks like a place name when query wants something else: -420
- result mostly matches only a place token: -420

For a one-token street search, the candidate must pass own-field verification: its street-name tokens must match one of the prepared direct, alias, substring, or fuzzy alternatives. A token that merely appears in the street's administrative context is not enough. Street-shaped tokens ending in forms such as **str**, **strasse**, **street**, **road**, **weg**, **gasse**, **allee**, **platz**, or **ring** also prevent an administrative area from taking the one-token street query merely because it shares a word.

### 17.3 Admin ranking

Admin score:

$$\begin{aligned} \text{score} = & \text{base} + 15 \cdot \text{matchedTokens} \\ & + 25 \cdot \text{nameMatches} + 8 \cdot \text{parentMatches} \end{aligned}$$

Here **base** is 135 for one-token place queries, 75 for other non-number queries, and 10 for number queries.

Additional terms:

- higher-level admin preference:  $\max(0, 14 - \text{adminLevel}) * 3$
- one-token match quality: four times the direct/prefix/substring/fuzzy token quality
- one-token, one-word admin name: +850
- exact one-token area at admin level 2–6: +2,600
- exact one-token area at admin level 7–8: +1,700
- exact one-token area below level 8: +200
- exact multi-token area: +2,600 through level 8, otherwise +1,700
- known place exact match through level 8: another +900
- known place full name-token match through level 8: another +900

One-token admin ranking also uses an admin-level score: +2,600 for levels 1–6, +1,150 for level 7, +200 for level 8, -900 for level 9, and -3,200 for more detailed levels. A bounded importance term combines admin level and bounding-box span, capped at 1,000 for an exact name and 700 otherwise. These terms prefer recognizable countries, states, regions, and cities over a tiny administrative feature with the same partial text. They are only applied in the one-token path, where this ambiguity is strongest.

These terms let a direct city or region name outrank an incidental POI that contains the same word outside the matching administrative context.

## 17.4 POI ranking

POI score:

$$\begin{aligned} \text{score} = & \text{base} + 15 \cdot \text{matchedTokens} \\ & + 30 \cdot \text{poiMatches} + 12 \cdot \text{adminMatches} \end{aligned}$$

Here **base** is 20 for number queries and 105 otherwise.

Additional terms:

- exact POI display name: +55
- exact normalized POI label, name, or brand in text relevance: +900
- complete query phrase contained in label, name, or brand: +520
- each query token matching name or brand: +120
- each query token matching any POI field: +35
- all query tokens present in name or brand: +420
- POI query type score: favors the category implied by the query
- map-center focus score: up to +180 based on distance
- inside current viewport: +160
- one-token match quality: twice the direct/prefix/substring/fuzzy token quality
- known place query and POI admin matches that place: +420
- known place query and POI admin does not match: -1900

The POI type score contains explicit protections for common category collisions. Parking or parking-space POIs receive -700 unless the query asks for parking; playgrounds receive -500 unless the query asks for a playground. Fast-food POIs receive +320 and restaurants or cafes receive +180 in the general deterministic POI ranking. Natural-language concept ranking adds its own validated category weight later.

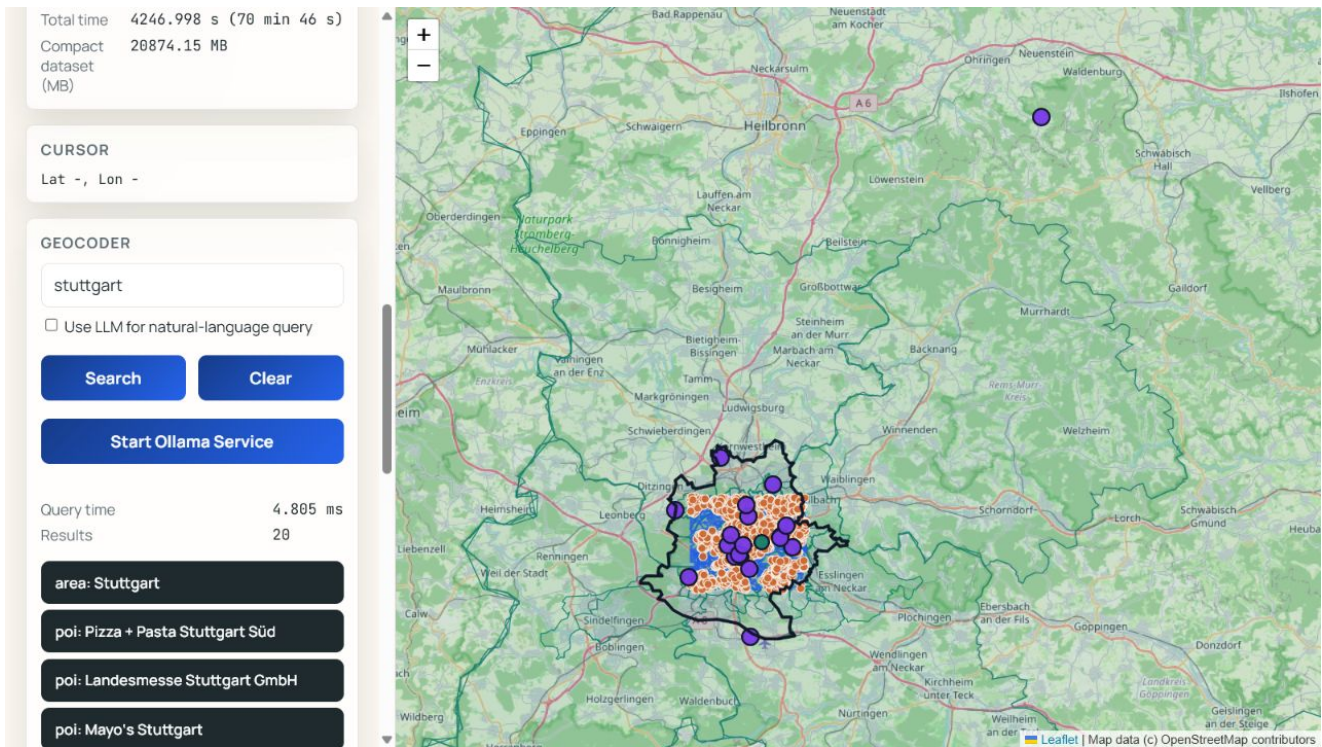


Figure 17: A direct place-name search: the matching admin area is ranked before incidental POI text matches.

## 17.5 Additional candidate helpers

The geocoder can add candidates after the strict intersection:

- **street-place helper:** splits an ordered query into street and place portions, verifies that the place portion identifies an admin area, then checks street-owned tokens and linked admin tokens
- **brand-place helper:** tries both possible sides of each ordered split as brand and place, requires the place side to resolve to an admin area, intersects POI-owned brand/name/category postings, then scores brand-field and admin matches
- **English/German place aliases:** deterministic pairs are limited to **Vienna/Wien**, **Zurich/Zuerich**, **Munich/Muenchen**, **Cologne/Koeln**, and **Nuremberg/Nuernberg**
- **missing-e German variants:** for one-token queries, up to four alternatives insert **e** after **a**, **o**, or **u**; later field verification decides whether the alternative is useful

These helpers still return only records from the local PBF-derived dataset.

The helper loops have explicit work limits. Street-place lookup rejects an anchor posting above 250,000 references and collects at most 6,000 helper candidates. Brand-place lookup rejects an anchor above 1,200,000 references and normally visits at most 90,000 references. These are latency guards; they can stop an extremely broad query from exploring every possible interpretation.

## 17.6 Result-type filters, final sorting, and ties

The optional **type** parameter is applied after candidates have been generated and scored. Supported values map to house, street, admin area, and POI. With no filter, all surviving types compete in one list. With a filter, candidates of other types are removed and the remaining candidates keep their internally calculated scores.

Final sorting is deterministic:

1. higher score first

2. for equal scores, packed type order: house, street, admin area, POI
3. for equal score and type, smaller record index first

The record index is stable for one loaded snapshot. It is not a statement that a lower OSM ID is more important; it is only a repeatable final tie-break. The list is truncated to the requested `limit` after sorting.

## 17.7 What ranking can and cannot infer

Ranking resolves evidence available in the local snapshot: names, brands, categories, address fields, admin links, geometry, and map context. A short fragment can remain genuinely ambiguous. For example, a fragment may be both the beginning of a city and part of a business name. The scoring rules prefer strong own-field matches and important admin areas, but they cannot read an unstated intention from the user's mind. Selecting Houses, Streets, Admin, or POIs removes one source of ambiguity without changing the stored data.

## 18 Reverse geocoder

### 18.1 Input and output

Reverse geocoding receives a coordinate:

```
/api/reverse?lat=48.7758&lon=9.1829
```

It returns:

- whether something was found
- closest house when available
- closest street when available
- matched admin areas
- distance values
- query time

### 18.2 Distance search

The reverse geocoder checks nearby spatial grid cells. It does not scan all houses and all streets.

House and POI distance uses the haversine formula between two coordinates. Street distance projects the clicked point onto every segment of a candidate street polyline and keeps the shortest point-to-segment distance. Admin containment first checks candidate polygons in the clicked grid cell; when no containing area is found, nearby admin cells are expanded and distance to admin geometry is considered.

The primary reverse result follows explicit distance thresholds:

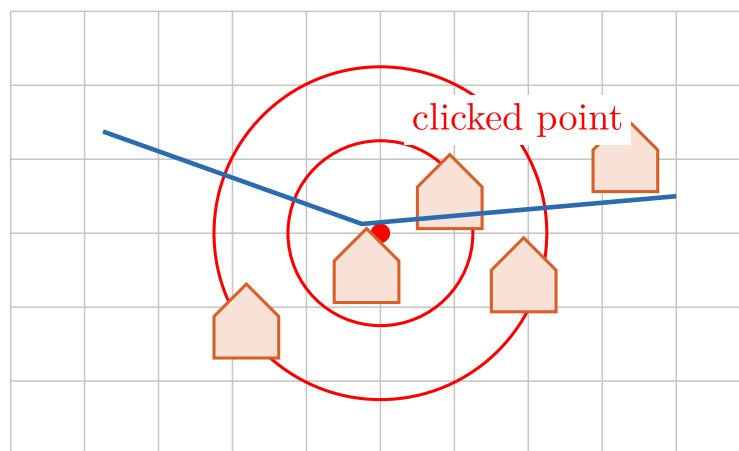
1. house within 3 m
2. street within 15 m
3. street within 50 m when it is no farther than the nearest house
4. house within 100 m
5. any remaining street within 50 m
6. house within 300 m

7. containing or nearest admin area
8. otherwise a street or house within 1,000 m

The response can still include the nearest house and nearest street as supporting fields even when a different type becomes the primary result. For containing admin areas, the server constructs a coherent parent chain and prefers the most detailed member as the primary admin result. Inner-ring holes are respected by the same admin containment routine used during parsing.

### 18.3 Reverse-result drawing

The browser draws the clicked coordinate, the selected primary result, supporting nearest objects, and the point-in-polygon admin chain. When the primary result is a house or street, a dashed line connects the click to that object. The primary result is drawn larger and darker than supporting objects. This visual distinction explains why the panel can list both a closest house and a nearest street while naming only one as **Result type**.



Search expands through nearby grid cells until enough candidates are checked.

Figure 18: Reverse lookup expands around the clicked point instead of scanning the full dataset.

## 19 Natural-language and Ollama mode

### 19.1 What Ollama is allowed to do

Ollama is used only for intent parsing. It does not return final places. The backend still searches local OSM indexes and validates the result.

The LLM can produce these intent types:

- named POI or brand in a place
- nearest category to address
- nearest product/service concept to address or viewport
- plain address-shaped query

### 19.2 Two verification passes

The program asks Ollama to draft strict JSON, then asks it to verify/correct that JSON twice. All three calls use JSON output mode, temperature 0, top-p 1, seed 0, and a maximum of 120 predicted



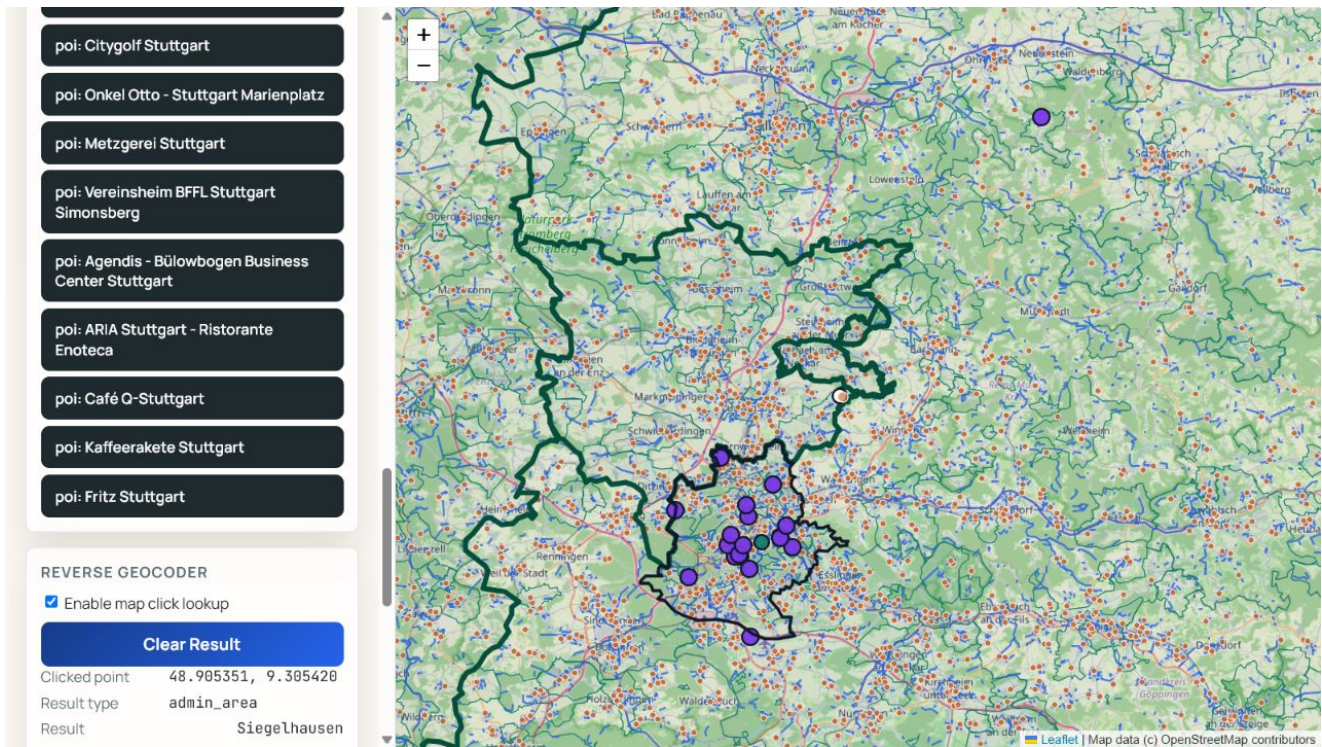


Figure 19: A reverse lookup in the running browser UI.

tokens. These settings reduce sampling variation, but they do not mathematically turn the model into a deterministic parser across every runtime and model build.

Verification pass 1 receives the original sentence and draft JSON. Verification pass 2 receives the first corrected JSON, or the draft when pass 1 returned nothing. The final JSON is pass 2 when available, otherwise the latest available JSON. The backend records how many verification calls returned content. `llmVerified` is true only when both verification responses exist and the parsed intent is supported.

The final JSON is parsed by the backend. Invalid or unsupported JSON does not directly become a result. The accepted C++ intent is kept separately from every model response.

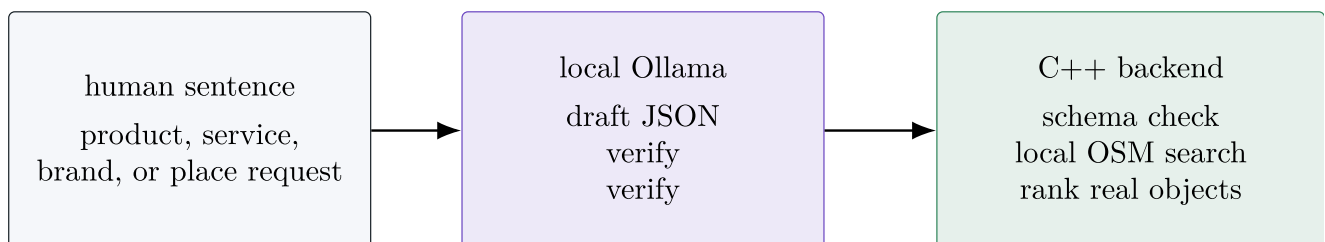


Figure 20: The LLM translates intent; the server decides using local OSM data.

### 19.3 Natural address guard

Short address-shaped input is handled by the structured address planner even when an LLM draft labels it as a POI request. The planner separates the number, recognized administrative context, and remaining street phrase before it searches. This keeps an address from entering the brand or product path merely because the model selected the wrong intent name.

The short-address shape is deliberately narrow: exactly one number token, no postcode, and two to four text tokens. Product/service sentences do not satisfy this shape. The deterministic address planner runs first. A street-level result is allowed only when a real matching street is found in the resolved place and no exact stored house satisfies the complete address.

## 19.4 Deterministic intent precedence

Before calling or accepting Ollama, the backend also parses recognizable query shapes deterministically. This is important for exact brands, supported nearby categories, and known product/service phrases.

After the Ollama calls:

- a deterministic named-brand/POI intent replaces a conflicting non-brand LLM intent
- punctuation-sensitive deterministic brand text containing `&`, apostrophe, or hyphen is preserved
- a deterministic category replaces a conflicting category intent
- a deterministic product/service concept replaces a conflicting concept intent unless Ollama produced a named-brand intent
- if Ollama produced no supported intent, the deterministic intent is used
- if neither route understands the query, the endpoint returns HTTP 400 with `query intent not understood`

This precedence does not create map objects. It decides which already implemented backend search path receives the query. The Ollama draft and verification JSON remain visible in diagnostics even when C++ accepts a stricter deterministic intent, so the difference can be inspected.

## 19.5 Product and service concept graph

Product/service queries map user words to OSM tag families. Example:

- medicine or cough syrup can map toward pharmacy/chemist
- bread can map toward bakery/supermarket
- bike repair can map toward bicycle shop/service
- printing documents can map toward copyshop/print service

This does not prove inventory. It returns likely real OSM places based on category tags.

## 19.6 Concrete addresses, named places, and vague anchors

A natural nearest-category or nearest-concept intent needs an origin. The backend evaluates origins in this order:

1. a usable concrete address from the accepted intent
2. the same address without its house number when only the street can be resolved
3. the map-center latitude/longitude sent by the browser

When the accepted intent has no address, the backend searches the original sentence for an explicit loaded administrative place and can use that as the restriction. A place phrase in the user's sentence can therefore establish the spatial context even when the LLM left the address field empty.

The address validator rejects an LLM-generated anchor when it is only a vague phrase, when it is made entirely from generic words, when its words did not come from the user's query, or when it has no house number, postcode, or resolvable loaded place name. The response exposes `ignoredAddress` and `ignoredAddressReason`. A rejected phrase such as `some hotels` is not geocoded as if it were a street address.

If no valid address and no map center are available, the natural endpoint returns an empty HTTP 200 response containing `originSource=none`. This means the request was structurally understood but no spatial origin was available; it is different from an unrecognized intent, which returns HTTP 400.

## 19.7 Reference-category search step by step

The `referenceCategory` field represents a relationship between a target concept and a generic nearby category. It is recognized from supported category words handled by the parser.

1. Collect POIs whose stored category/tag matches the reference category.
2. If a viewport is available and the origin is not a concrete address, inspect reference POIs inside that viewport first.
3. Expand through POI-grid rings around the origin until enough anchors are collected or a guard stops the scan.
4. Keep at most 48 nearest reference anchors.
5. Use at most the first 12 anchors, plus the original origin, as target-POI search centers.
6. For every matching target POI, find its closest collected reference anchor.
7. Score the target using category/concept weight, text relevance, query-type relevance, reference proximity, viewport membership, and distance.

Reference collection inspects at most 40,000 grid cells, 300,000 POI records, and 512 temporary matching anchors. Target search inspects at most 160,000 grid cells and 750,000 distinct POI records. The response reports `poiRecordsInspected` and `poiSearchTruncated`; these reveal when a guard, rather than lack of local data, ended a broad scan.

Natural POI score is:

$$\begin{aligned} score = & 9 \cdot graphWeight + textRelevance + typeScore \\ & + \max(0, 620 - referenceDistance/2) \\ & + 260 \cdot inViewport - \min(520, distanceBasis/120) \end{aligned}$$

`distanceBasis` is distance to the closest reference anchor when one exists; otherwise it is distance to the resolved origin. After score, ties prefer larger graph weight, viewport membership, presence of a reference anchor, smaller reference distance, smaller origin distance, and finally smaller POI record index.

## 19.8 Local knowledge graph in depth

In this project, the knowledge graph is not a remote database and not a learned model. It is a local set of typed concepts and allowed connections used by the backend to turn messy product/service wording into OSM-searchable tags. Ollama may suggest an intent, but the backend checks the intent against this graph before it searches.

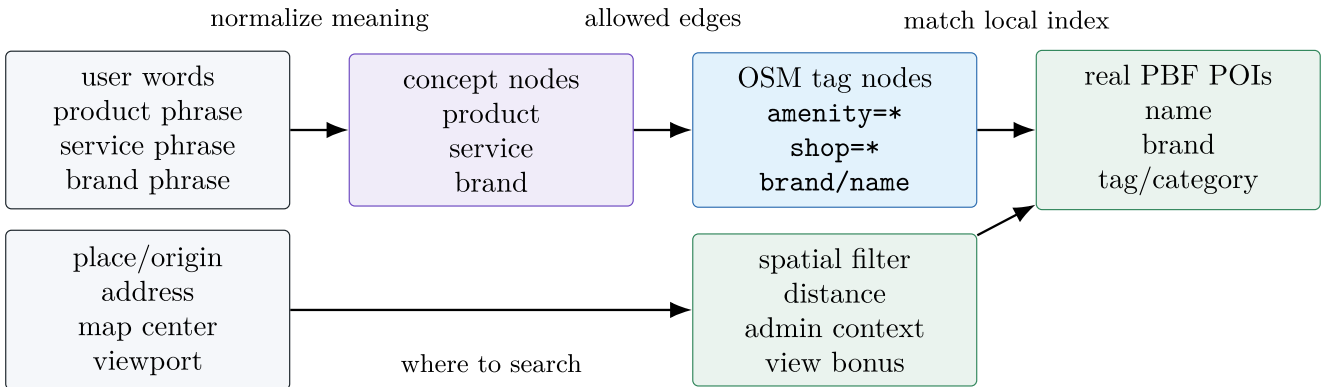


Figure 21: The local knowledge graph connects human words to allowed OSM tag families and then to real POIs from the loaded PBF.



## Graph node types

The graph uses several node types:

- **Product concept:** a thing the user wants to buy or find, such as medicine, bread, groceries, electronics, or fuel.
- **Service concept:** an action the user wants, such as bike repair, document printing, eating, charging, parking, or banking.
- **Brand concept:** a name that may appear in the stored POI **name** or **brand**, such as a restaurant chain, supermarket, car brand, parcel service, or hotel brand. An OSM operator can participate only when it became the selected display name during parsing.
- **Category concept:** a local category used by the program, such as restaurant, fast food, pharmacy, shop, hotel, fuel, park, or museum.
- **Place/origin concept:** the location context, which can come from a typed place name, an address, the current map center, or the current viewport.
- **OSM tag concept:** a searchable stored field such as `shop=*`, `amenity=*`, `tourism=*`, `leisure=*`, `landuse=*`, `name`, or `brand`.

## Graph edge meanings

Edges are not proof that a product exists at one exact shop. They are allowed search directions. For example:

- **product likely sold at category:** medicine can search pharmacies/chemists; bread can search bakeries or supermarkets.
- **service provided by category:** bike repair can search bicycle shops or repair-related POIs.
- **brand matches text fields:** a brand query searches the stored POI name and brand fields, plus category/tag text used by the primary index.
- **category maps to OSM tags:** fast food maps toward `amenity=fast_food`; hotel maps toward `tourism=hotel`; pharmacy maps toward `amenity=pharmacy`.
- **origin limits or biases results:** an address or map center gives the nearest-category search a starting point; a named place gives admin context.

## Complete concept catalogue

The final backend contains the following broad concepts. Numbers in parentheses are graph weights; 100 is the strongest category for that concept.

| Concept                 | Allowed stored OSM key/value targets                                                   |
|-------------------------|----------------------------------------------------------------------------------------|
| personal_care           | shop=chemist (100), shop=cosmetics (95), amenity=pharmacy (85), shop=supermarket (55). |
| medicine                | amenity=pharmacy (100).                                                                |
| bakery_food             | shop=bakery (100), supermarket (65), convenience (50).                                 |
| groceries               | supermarket (100), convenience (75), greengrocer (70), deli (60).                      |
| electronics_accessories | electronics (100), mobile phone (95), computer (90), supermarket (45).                 |
| clothing                | clothes (100), fashion (95), department store (75), sports shop (45).                  |
| shoes                   | shoes (100), sports shop (65), department store (55).                                  |

|                  |                                                                              |
|------------------|------------------------------------------------------------------------------|
| books_stationery | books (100), stationery (95), department store (45).                         |
| flowers          | florist (100), garden centre (80), supermarket (40).                         |
| bike_service     | bicycle shop (100).                                                          |
| car_service      | car repair (100), car parts (95), tyres (90).                                |
| fuel             | amenity=fuel (100).                                                          |
| restaurant       | restaurant (100), cafe (70), fast food (65).                                 |
| fast_food        | amenity=fast_food (100).                                                     |
| hotel            | tourism=hotel (100).                                                         |
| hospital         | amenity=hospital (100).                                                      |
| school           | school (100), university (60).                                               |
| park             | leisure park (100), garden (80), recreation ground (70), village green (70). |
| banking          | amenity=bank (100).                                                          |
| parking          | amenity=parking (100).                                                       |
| sports           | sports shop (100), sports centre (70), swimming pool (55).                   |
| toys_games       | toys (100), games (95), department store (50).                               |
| jewelry          | jewelry (100), watches (95).                                                 |
| hardware         | hardware (100), do-it-yourself (95), paint (85), garden centre (60).         |
| furniture_home   | furniture (100), houseware (85), interior decoration (80).                   |
| optician         | optician (100).                                                              |
| pet_supplies     | pet shop (100).                                                              |
| photo_printing   | photo (100), copyshop (95), stationery (50).                                 |
| hair_beauty      | hairdresser (100), beauty (90), cosmetics (60).                              |

## Complete product-family catalogue

Product families make one broad concept more specific. The family is checked before the broader concept, so `running_shoes` can prefer a shoe or sports shop differently from a general clothing request.

| Product family        | Broad concept           | Strongest target(s)                |
|-----------------------|-------------------------|------------------------------------|
| cosmetics_removal     | personal care           | chemist, cosmetics                 |
| hair_body_care        | personal care           | chemist, cosmetics, supermarket    |
| dental_care           | personal care           | chemist, pharmacy                  |
| baby_hygiene          | personal care           | chemist, supermarket               |
| cold_medicine         | medicine                | pharmacy                           |
| pain_allergy_medicine | medicine                | pharmacy                           |
| first_aid             | medicine                | pharmacy, chemist                  |
| bread_pastry          | bakery food             | bakery, supermarket                |
| cake_sweets           | bakery food             | bakery, confectionery              |
| daily_groceries       | groceries               | supermarket, convenience           |
| fruit_vegetables      | groceries               | greengrocer, supermarket           |
| pet_food              | pet supplies            | pet shop, supermarket              |
| phone_accessories     | electronics accessories | mobile phone, electronics          |
| computer_accessories  | electronics accessories | computer, electronics              |
| printer_supplies      | electronics accessories | computer, electronics, stationery  |
| outerwear             | clothing                | clothes, fashion, department store |
| basic_clothing        | clothing                | clothes, fashion                   |

|                      |                  |                                |
|----------------------|------------------|--------------------------------|
| running_shoes        | shoes            | shoes, sports shop             |
| books                | books/stationery | books                          |
| stationery           | books/stationery | stationery, books              |
| flowers_plants       | flowers          | florist, garden centre         |
| garden_supplies      | hardware         | garden centre, do-it-yourself  |
| diy_tools            | hardware         | hardware, do-it-yourself       |
| paint_supplies       | hardware         | paint, do-it-yourself          |
| bike_parts_repair    | bike service     | bicycle shop                   |
| car_parts            | car service      | car parts, tyres, car repair   |
| glasses_lenses       | optician         | optician                       |
| photo_documents      | photo/printing   | photo, copyshop                |
| hair_services        | hair/beauty      | hairdresser                    |
| beauty_products      | hair/beauty      | beauty, cosmetics              |
| toys                 | toys/games       | toys                           |
| games                | toys/games       | games, toys                    |
| jewelry_watches      | jewelry          | jewelry, watches               |
| watch_repair_battery | jewelry          | watches, jewelry               |
| home_furniture       | furniture/home   | furniture, interior decoration |
| home_textiles        | furniture/home   | houseware, furniture           |
| sports_equipment     | sports           | sports shop                    |

The deterministic phrase recognizer maps English/German phrases to these exact families. Examples include:

- shampoo; cough syrup/Hustensaft; bread/Brot
- phone charger/Ladegeraet; bicycle repair/Fahrradreparatur
- printer ink/Druckerpatronen; passport photos/Passfotos
- watch battery/Uhrenbatterie

Ollama is prompted with the same allowed concept and family names. The backend normalizes and validates the returned names; an arbitrary model-created family has no graph rule and therefore no positive tag weight.

### How Ollama output is checked against the graph

Ollama receives the sentence and is asked for strict JSON. The backend does not accept free text. The JSON must name an allowed intent, such as named POI/brand search, nearest category to address, nearest concept to address or viewport, or address-shaped lookup.

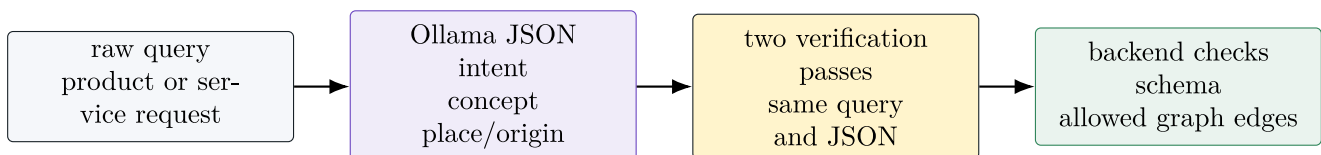


Figure 22: Ollama proposes structured intent; the backend verifies the shape and rejects unsupported graph paths.

The backend checks:

- whether the intent name is supported

- whether required fields are present
- whether the concept/category can be mapped to local OSM tags
- whether a place or address origin can be resolved by the local geocoder
- whether the final candidates exist in the loaded PBF

If these checks do not establish a supported local search, the program does not invent a result. The request is rejected or returns no matching candidates. A street-level response is produced only by the separate structured-address path when a real street matches but no exact stored house does.

### Candidate selection from graph-derived tags

After validation, the graph produces search handles. These handles are not final answers. They are only candidate selectors.

1. A product/service concept chooses allowed categories and tag families.
2. A brand concept chooses stored text fields: POI name and brand, with category/tag text available for validation and ranking.
3. A place or address chooses an origin and admin context.
4. The server searches POI indexes and spatial grids for candidates.
5. Each candidate is still a real OSM object from the binary snapshot.

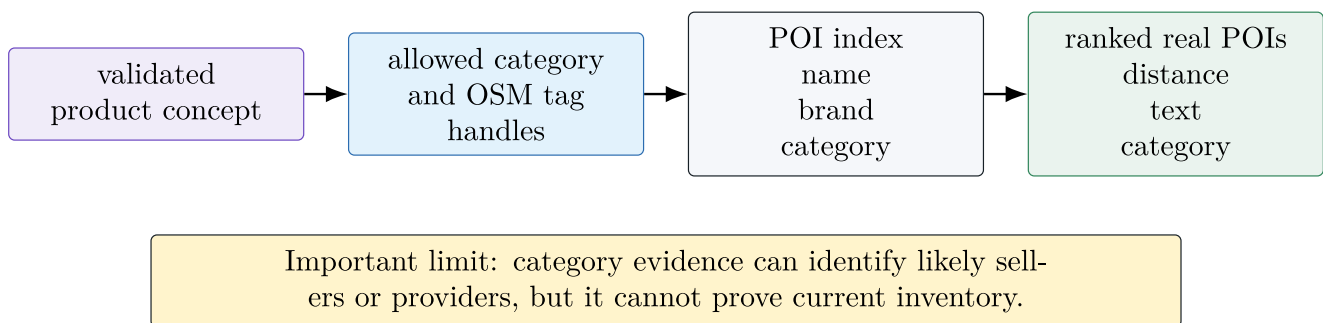


Figure 23: Graph-derived tags select candidate POIs; ranking decides which real local objects appear first.

### How graph relevance enters ranking

The ranker combines graph relevance with the normal geocoder signals:

- **text relevance:** the candidate name, brand, category, or tag value matches the query words.
- **graph relevance:** the candidate category/tag is one of the allowed graph targets for the concept.
- **admin relevance:** the candidate lies inside or near the named place context.
- **distance relevance:** nearest-category searches prefer candidates closer to the resolved origin.
- **viewport relevance:** ambiguous searches can prefer candidates near the current map view.
- **exact brand relevance:** a direct brand/name match beats a broad category match.

A direct stored brand/name match is therefore narrower and stronger than a broad product or category phrase. Place context further raises candidates linked to the requested administrative area.

## What the graph deliberately does not do

The graph does not call the internet, does not query store catalogues, does not check opening hours in real time, and does not create synthetic POIs. It only changes how the local OSM data is searched and ranked. The final visible result must still be a real object stored in the loaded binary snapshot.

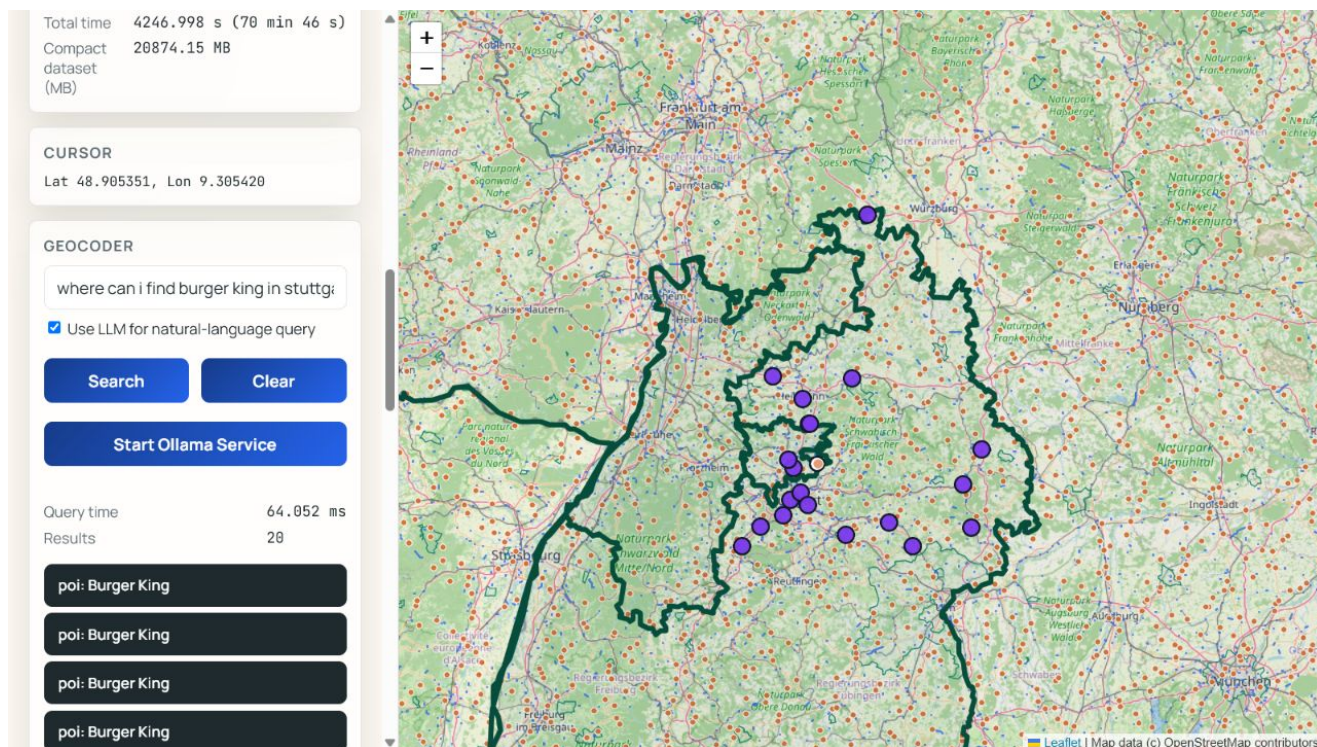


Figure 24: LLM mode in the browser: the natural sentence becomes local OSM POI results.

## 20 Frame of reference

### 20.1 Map center

The frontend sends map center coordinates with geocoder requests. This lets ranking favor nearby POIs when the query is ambiguous.

### 20.2 When place names override the current view

If the query clearly names a known place, the program does not trap the result inside the current view. The explicit place becomes the search context and can move the result beyond the visible map rectangle.

### 20.3 When the current view helps

When no explicit place or address is present, the current view can raise nearby matching POIs above equally relevant distant ones.

## 21 Browser controls

### 21.1 Geocoder text field

The text field sends to:

- `/api/geocode` when LLM checkbox is off
- `/api/natural-geocode` when LLM checkbox is on

Pressing Search sends at most 20 requested results. In natural mode the browser also sends the current map center and all four viewport bounds. Pressing Enter inside the text field triggers the same search action. Clear removes geocoder layers and resets the result panel; it does not unload the snapshot or clear server indexes.

## 21.2 All, Houses, Streets, Admin, and POIs

These buttons select the server-side `type` parameter. All leaves the parameter empty and preserves mixed ranking. A selected type does not run a separate database or rebuild an index; it asks the same geocoder to retain only one object type after scoring. Natural nearest-category and nearest-concept searches return POIs, so selecting a non-POI type for those intents produces an empty typed result rather than relabeling a POI as a house or street.

## 21.3 Start Ollama Service button

The frontend includes a button to start the local Ollama service. The backend accepts this operation only for a local Ollama host. It finds processes owning the configured TCP port, terminates those port owners, starts `ollama serve` in its own process group, waits for the service to answer, and sends a short warm-up request for the configured model. The browser reports the host, port, warm-up status, and number of cleared port processes.

At normal server startup, automatic Ollama launch can be disabled with `OSM_AUTO_OLLAMA=0`. When this program starts its own managed Ollama process, server shutdown stops that managed process group. An Ollama process that was already healthy and reused at startup is not treated as owned by this server.

## 21.4 Reverse geocoder checkbox

When enabled, map clicks call `/api/reverse`. When disabled, clicking the map does not run reverse lookup.

## 21.5 Viewport refresh and stale requests

Map movement schedules a delayed refresh instead of issuing a request for every movement event. When a newer viewport refresh begins, the frontend aborts the previous fetch group and uses a request sequence number so an older response cannot overwrite a newer map state. Houses, streets, and admin areas are fetched concurrently. Street and admin limits grow with zoom level, while the house slider remains directly controlled by the user.

The map's OSM base tiles, Leaflet JavaScript/CSS, and web fonts are external browser resources in the current HTML. Geocoding data and search results remain local, but the visual base map and those frontend libraries require network access unless they are already cached.

# 22 Performance controls

## 22.1 Progress bars

Parser and index phases print progress bars with:

- phase number
- phase name

- percentage
- completed bytes or records
- speed
- elapsed time
- ETA when enough progress exists

## 22.2 Why ETA can be unknown

At the very beginning there is not enough completed work to calculate a meaningful speed. ETA becomes meaningful only after completed units and rate are both non-zero.

## 22.3 Why speed can drop during parsing

The displayed speed is not pure disk speed. A phase may start with fast sequential decompression and later slow down when it performs more filtering, node lookup, geometry extraction, string interning, or memory writes.

The byte rate shown inside one PBF pass is cumulative bytes reported by the current reader divided by elapsed time for that pass. It is not instantaneous SSD throughput. Early cached reads can make the first displayed average very high; later decompression and object handling increase elapsed time and lower the average even when the program is healthy.

The parser's final `totalSeconds` is the sum of the rounded relation scan, extraction, relation assembly, admin attribution, and optional street-connection components. Forward-index construction and binary-snapshot writing happen afterward and have their own progress output; they are not silently folded into that PBF parse total. Input MB/s is PBF file size divided by the parser component total, while objects/s uses scanned primitive counters divided by the same total.

Peak RSS is an operating-system process measurement sampled at important parser boundaries. The `datasetMB` value is a capacity-based estimate calculated from owned vectors and strings, including the forward index when present.

These two values answer different questions: RSS is observed process residency, while dataset MB estimates the memory owned by the compact data structures.

## 23 Memory controls

### 23.1 Released string interner index

During parsing, `StringInterner` needs a hash map to avoid duplicate strings. After strings are released into `data.strings`, the hash map index is cleared before admin attribution. This lowers peak memory during the expensive geometry lookup phase.

### 23.2 Flat relation geometry

Relation member geometry uses flat spans plus a coordinate vector instead of a map from way ID to a vector. This reduces small allocations and duplicate vector overhead.

### 23.3 Moved posting lists

Forward index finalization moves posting vectors from temporary maps into final posting lists. It does not copy every posting vector again.

## 23.4 Packed geocode refs

GeocodeRef is packed to 5 bytes:

- 1 byte type
- 4 bytes record index

This matters because Europe has many posting references.

## 24 How query shapes select a search path

The geocoder does not treat every input as one flat text string. It chooses a path from the structure that is present:

- A number plus a street phrase and place or postcode enters structured address resolution.
- A street phrase plus a place enters street-and-place matching.
- A single place phrase gives exact admin names and direct object names priority over objects that merely contain that place in their context.
- A named POI or brand plus a place searches indexed POI text and verifies the place through admin links.
- A product or service sentence enters the natural concept graph only when LLM mode is enabled.
- A target concept relative to a generic category enters the reference-anchor search path.

These paths share the same stored objects and indexes. The difference is which fields are required, which object types are eligible, and which ranking signals are allowed to influence the result.

## 25 Complete implementation walkthrough

This section is the slow, detailed version of the program. It explains each feature as a mechanical process: what the feature means, what input it reads, what data is accepted, what data is rejected, what arrays or indexes are changed, and what the browser or API can do with the result.

### 25.1 PBF primitives in the program

OpenStreetMap stores map data as three primitive types.

- A **node** is one coordinate pair. The parser uses nodes for address points, POI points, and geometry points used by ways.
- A **way** is an ordered list of node IDs. The parser uses ways for roads, building outlines, POI areas, and simple admin boundary lines.
- A **relation** is a list of other nodes, ways, or relations with roles. The parser uses relations for administrative multipolygons and addressed building relations.

The important detail is that a way usually stores only node IDs, not the coordinates of those nodes. That is why the parser cannot read Europe in one simple pass and immediately know every road shape. It first learns which node IDs it needs, then scans the nodes to recover their coordinates.



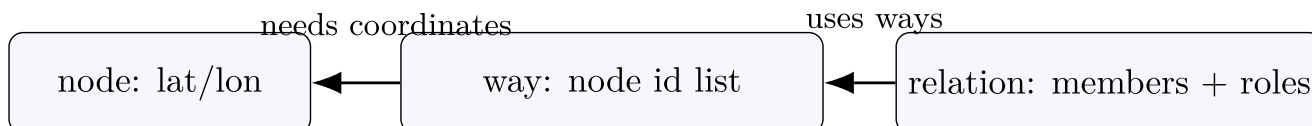


Figure 25: A road or boundary way points to nodes. The parser must recover node coordinates before it can draw geometry.

## 25.2 Tag reading and name selection

Every OSM object has a small dictionary of tags. The parser does not copy all tags into the final dataset. It reads tags only long enough to decide whether an object matters and which compact fields should be stored.

| Tag group                                                                                                        | How the program uses it                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>addr:*</code>                                                                                              | Builds house/address records. <code>addr:housenumber</code> , <code>addr:street</code> , <code>addr:postcode</code> , <code>addr:city</code> , and <code>addr:country</code> become string references in <code>HouseRecord</code> . |
| <code>name</code> , <code>name:de</code> , <code>name:en</code>                                                  | Chooses display/search names. The selected language is tried first, then plain <code>name</code> , then selected <code>official_name</code> , plain <code>official_name</code> , and finally road <code>ref</code> when needed.     |
| <code>highway</code>                                                                                             | Decides whether a way is a street and whether it is a real line road or an area/path that should be skipped.                                                                                                                        |
| <code>boundary=administrative</code> ,<br><code>admin_level</code>                                               | Decides whether a way or relation is an administrative area. Levels outside the accepted integer range are ignored.                                                                                                                 |
| <code>amenity</code> , <code>shop</code> , <code>tourism</code> ,<br><code>leisure</code> , <code>landuse</code> | Decides whether an object becomes a searchable POI and which local category it receives.                                                                                                                                            |
| <code>brand</code> , <code>operator</code>                                                                       | Brand is stored independently and can match chain queries. Operator is used only as a name source when stronger name fields are absent and is not retained as an independent field.                                                 |

The code uses normalized search text only for indexing and matching. The original readable string remains in the string table so the browser can show the real name with accents and punctuation.

## 25.3 Address extraction from nodes

A node becomes a house record when it carries address tags. The parser checks the tags first, then stores the coordinate directly because a node already has latitude and longitude.

1. Read a node from libosmium.
2. Check whether `addr:housenumber` or `addr:street` exists.
3. If neither field exists, the node is not an address and is skipped for house extraction.
4. If at least one address field exists, copy the coordinate into a `HouseRecord`.
5. Store address strings through the `StringIntern`; the record keeps integer references, not full string copies.
6. Mark the source as `node` so the summary can later say how many houses came from nodes.

This means a partial address is still stored. For example, a node with a street but no house number is useful for map display and reverse geocoding, even if it cannot satisfy an exact house-number query.

## 25.4 Address extraction from building ways

A building way is a polygon or outline made from node IDs. If the building has address tags, the parser stores one representative coordinate instead of keeping the whole building shape as a house. This keeps the final dataset small and makes point search fast.

1. Read a way and inspect its tags.
2. Require address tags and a building-like tag for addressed building extraction.
3. Add all node IDs from this way to the needed-node store.
4. During the node-coordinate recovery scan, fill the coordinates for those node IDs.
5. Compute a representative point from the available coordinates.
6. Store one **HouseRecord** with source **way**.

The representative point is enough for this program because the browser displays address dots and the reverse geocoder needs nearest address points. The full building polygon is not needed for the implemented geocoder behavior.

## 25.5 Address extraction from building relations

Some buildings are relations instead of plain ways. The parser handles addressed building relations separately because their members may be several ways.

1. The relation scan records building relations with address tags.
2. It remembers which member ways are needed for those relations.
3. When those member ways are later seen, their node IDs and coordinates are collected.
4. After way geometry exists, the parser computes one representative house coordinate for the relation.
5. The relation becomes a **HouseRecord** with source **relation**.

This avoids losing addresses that are modeled as multipolygon buildings.

## 25.6 Street extraction and street skip rules

A street is stored from a way when the way has an accepted **highway** value. The parser does not store every line-like object because Europe contains many paths, platforms, corridors, and proposed roads that would increase memory while making address search noisier.

1. Read a way and check **highway**.
2. Reject it if there is no **highway** tag.
3. Reject it if **area=yes**, **area=1**, or **area=true**; those are surfaces, not road centerlines.
4. Reject construction, proposed roads, paths, footways, cycleways, bridleways, steps, pedestrian areas, platforms, and corridors for the main street set.
5. Choose the street display name using the language/name rule.
6. If the street has a name or usable reference, store it as a named street.
7. If it has no name, store it only when unnamed connector recovery is enabled and the highway type is useful for connecting named road pieces.

The output is a **StreetRecord**. The record stores a name reference, a bounding box, a geometry offset, a geometry size, and a source object ID. The actual coordinates live in the shared **streetGeometry** array.

## 25.7 Unnamed street recovery

Many roads are split into several OSM ways. Sometimes a small connector piece has no name, even though it connects two named pieces of the same street network. If the program ignored all unnamed connectors, reverse geocoding and map display would show small gaps.

The parser keeps only selected unnamed connector types: link roads, residential connectors, living streets, unclassified roads, ordinary roads, and roundabout-like connector ways. It still skips footpaths, cycle paths, platforms, and proposed roads.

## 25.8 Street connection in full detail

Street connection is the process that merges road fragments that touch each other. It is used after street extraction, when all street records and their geometries already exist.

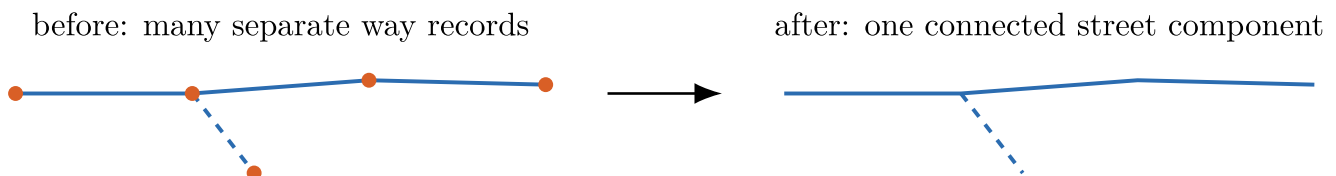


Figure 26: Street connection groups fragments when their endpoints touch. The geometry remains real OSM geometry; the program only groups matching pieces.

The process is:

1. For every street record, read the first and last coordinate from `streetGeometry`.
2. Skip records without a name, records with fewer than two points, and closed loops whose first and last coordinates are identical.
3. Create two endpoint records per remaining street. Each record contains the exact E7 integer coordinate, street-name string reference, highway-type string reference, and street index.
4. Sort endpoint records by name, highway type, latitude, longitude, and street index.
5. Create a disjoint-set structure. At the start, every street record is its own component.
6. For each equal run of endpoint records, union the street indexes. Equality here means the same name reference, same highway-type reference, and exactly equal stored endpoint coordinate.
7. Group street records by disjoint-set root.
8. Inside one component, build a local graph whose vertices are endpoint coordinates and whose edges are source street records.
9. Begin a chain at a vertex whose degree is not two when possible. This stops a branch from being flattened into an invalid single line.
10. Follow unused edges through degree-two vertices. Reverse a source geometry when the chain enters through its last endpoint, and skip the repeated junction coordinate when appending the next segment.
11. Store each completed chain as a new street record. Its name and highway type come from the first source record, its OSM ID is the minimum source OSM ID, and its bounding box is recomputed.
12. Copy closed loops and isolated records unchanged into the new arrays.
13. Replace `data.streets` and `data.streetGeometry` with the rebuilt arrays.

The merge therefore requires exact endpoint equality and exact stored name/highway identity. It does not snap nearby endpoints, infer that differently named roads are equivalent, or merge across a junction merely because two coordinates are close. The browser draws the final merged chains, not a hidden copy of the original street records.

The reason this is done after extraction is simple: the parser needs all streets first. A road segment at the beginning of the file might connect to a segment that appears much later in the PBF. Admin attribution runs after connection so links point to final street indexes. The forward index is built after that final street list exists.

## 25.9 POI extraction step by step

A POI is a searchable place such as a restaurant, shop, fuel station, hotel, park, pharmacy, or museum. The program stores only node and way POIs that have an accepted local category and a coordinate. Administrative and building relations are processed for their dedicated purposes; the final POI extraction path does not create relation POI records.

1. Read node or way tags.
2. Look for accepted POI tags: selected **amenity**, most **shop** values, selected **tourism**, selected **leisure**, and selected **landuse**.
3. Convert the OSM tag into a compact category such as restaurant, shop, hotel, park, pharmacy, or fuel.
4. Choose a display name from localized/name, then localized/brand, then localized/operator. A missing display name does not erase the accepted category/tag fields.
5. Store one coordinate: direct node coordinate for node POIs or a representative coordinate for way POIs.
6. Store name, brand, category, tag key, and tag value through the string table. There is no separate operator field in `PoiRecord`; an operator can become the selected display name when name and brand are absent.

The program does not store shop inventories. A drugstore POI can support a query for a likely drugstore product, but the data cannot prove that the exact product is on the shelf today.

## 25.10 Administrative boundary extraction

Administrative areas are countries, states, districts, cities, municipalities, and smaller named administrative regions. They are needed because addresses and streets are much clearer when the program knows which city or region contains them.

1. During the relation scan, keep relations with **boundary=administrative**.
2. Parse **admin\_level**; reject invalid or out-of-range levels.
3. Record relation members with their roles, especially **outer** and **inner**.
4. When member ways are seen, store their coordinates in flat relation-way storage.
5. Assemble member way pieces into closed rings.
6. Store outer rings as the visible containing area.
7. Store inner rings as holes that subtract from the containing area.

Each final admin area has a bounding box, name, admin level, source ID, and ring metadata. The ring metadata is important because a multipolygon can have more than one outer ring and can contain holes.

## 25.11 Inner holes in admin areas

An inner ring means: this part lies inside the outer outline, but is not part of the admin area. The point-in-polygon logic therefore uses two tests.

1. A point must be inside at least one outer ring.
2. The same point must not be inside any inner ring assigned to that outer ring.

### outer ring with inner hole

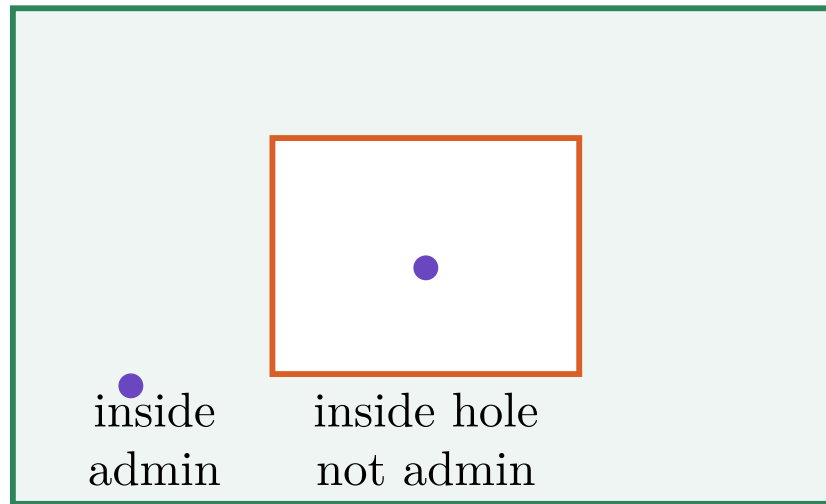


Figure 27: The outer polygon alone is not enough. The hole must be stored and subtracted.

## 25.12 Admin containment and object links

After admin geometry is assembled, the parser links houses, streets, POIs, and admin areas to the admin areas that contain them.

The process is:

1. Use the object's coordinate or representative coordinate.
2. Look up candidate admin areas through bounding boxes and grid pruning.
3. Run point-in-polygon only for candidates whose bounding box can contain the point.
4. Accept the admin area only when outer/inner ring logic says the point is truly inside.
5. Write compact links into arrays such as house-admin links and street-admin links.
6. Use those links later for display text, reverse geocoding, and ranking.

The program keeps links as arrays instead of storing a vector inside every object. That matters for Europe because millions of objects may have several admin ancestors.

## 25.13 Reverse geocoder step by step

Reverse geocoding starts with a clicked map point and answers: what is nearest here?

1. The browser sends latitude and longitude to `/api/reverse`.
2. The server checks nearby grid cells for house points, POI points, and street geometries.
3. For houses and POIs, distance is point-to-point.
4. For streets, distance is point-to-line-segment along the stored road geometry.
5. The nearest candidate of each type is kept.
6. Admin containment is looked up for the clicked point.

7. The JSON response returns the clicked coordinate, nearest objects, distances, and admin context.

The grid is what keeps this fast. The server does not scan every European street for every click.

### 25.14 Forward geocode index construction

The forward geocoder turns names and address fields into posting lists. A posting list is a list of object references that contain one normalized token.

1. Read every house, street, admin area, and POI.
2. Build normalized tokens from searchable fields.
3. Add object references into a temporary context map and a temporary primary map.
4. Move each map into sorted `GeocodePostingList` arrays.
5. Merge posting-list tokens into a sorted vocabulary.
6. Build a suffix array over vocabulary tokens for substring search.
7. Store the complete forward index inside the binary snapshot.

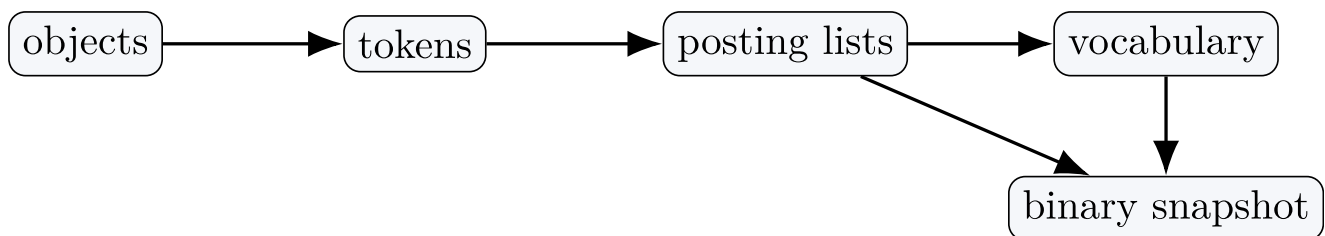


Figure 28: The forward index is built once during parsing and written into the binary snapshot.

### 25.15 Forward search query path

When the user searches without LLM mode, the program follows this path.

1. Normalize the query: lowercase ASCII, convert German umlauts and the German sharp s (ß) into searchable ASCII forms, discard combining-mark bytes handled by the normalizer, and split other punctuation or unsupported non-ASCII bytes into spaces.
2. Split tokens into ordered text tokens, house-number tokens, and postcode tokens.
3. For a numbered query, resolve exact administrative-place suffixes and treat the remaining ordered text as the street phrase.
4. Reject a number-plus-place query when no street phrase remains; it is incomplete rather than a valid address.
5. Build token alternatives for English/German place spellings where the implementation has deterministic rules.
6. Find posting lists for the query tokens.
7. For one-token queries, gather bounded substring and fuzzy alternatives even when a direct token exists; for multi-token queries, expand a token when its direct posting is missing.
8. Merge direct, alias, substring, and fuzzy posting references, then sort and deduplicate them by object type and record index.
9. Pick the smallest posting list as the anchor.
10. Intersect candidates against the other required token posting lists.

11. Verify one-token candidates against their own object fields so an admin-context-only coincidence cannot survive.
12. Add bounded street/place, brand/place, structured-address, and supported English/German alias candidates when their preconditions hold.
13. Score each surviving candidate by object type and query shape.
14. Apply the optional result-type filter, sort by score/type/index, then return the top results as JSON.

This is why query time stays low: the expensive work is the index build, not a full object scan on every search.

### 25.16 House ranking details

A house result is strong only when the query looks like an address. The ranking intentionally requires the house number to match if the user typed a number.

- Matching house number gives the largest address-specific boost.
- Matching postcode gives another strong boost when a postcode is present.
- Street-name token matches increase the score.
- City/admin token matches make the address more precise.
- If the user typed a number and the candidate has a different number, the candidate is rejected rather than weakly ranked.

These requirements prevent a different house number or a house on the wrong street from being presented as an exact address.

### 25.17 Street ranking details

A street result is useful when the query asks for a street or when a specific house number is missing from OSM.

- Exact street-name phrase matches receive a strong boost.
- Street token matches count more than unrelated admin token matches.
- If the query looks like a known place name, street results that only contain that place token are penalized.
- If a query contains a number, a street result can appear in the mixed or street-only view only when the exact house point is unavailable and the street/place pair is valid.

The response labels this outcome as street-level. The house-only view remains empty because a street coordinate is not an address point. No coordinate is interpolated along the street.

### 25.18 Admin ranking details

Admin areas are ranked strongly for place-name queries.

- Exact admin-name match receives a large boost.
- Known-place queries receive an extra admin boost.
- Lower admin levels that represent larger or more recognizable areas are considered, but name match is still the main signal.

- POIs that happen to contain the same word are penalized when they are outside the matching admin context.

This allows a direct administrative place-name match to outrank an incidental POI text match elsewhere in the dataset.

## 25.19 POI ranking details

POI ranking combines text relevance, category relevance, stored brand relevance, admin context, and map focus.

- Exact POI name matches are strong.
- Stored brand matches help chain queries such as fast-food or supermarket names. An operator helps only when parsing selected it as the POI display name.
- Category matches help natural queries such as **where can I find a pharmacy**.
- Admin context helps when the query includes a place name.
- The current map view can help nearby POIs, but a named place in the query can override the current view.
- A selected result is highlighted differently in the browser so the clicked ranked result is not confused with the other markers.

The ranking does not invent a place. A brand result is returned only if an OSM object in the loaded PBF has matching local data.

## 25.20 Natural-language search path

Natural-language mode is used only when the checkbox is enabled. It handles product, service, category, named-POI, brand, address, and relative-location sentences in English or German.

1. The browser sends the raw query, map center, and viewport to `/api/natural-geocode`.
2. The backend first runs deterministic guards for query shapes that should not need an LLM, such as short address-like queries.
3. If Ollama is enabled and reachable, the backend asks for strict JSON intent, not free prose.
4. The backend asks Ollama to verify the generated JSON against the original user query.
5. The backend repeats verification a second time.
6. The backend validates the JSON schema and allowed intent names.
7. The backend corrects or rejects unsafe/unsupported fields.
8. The backend executes the intent using only local OSM indexes and spatial grids.

Ollama is therefore not the geocoder. It is a parser for messy human wording. The actual results still come from the local PBF snapshot.

## 25.21 Natural address guard

Some address-like queries are short and should not become brand or POI searches. The backend detects this shape before the named-POI path.

- The query has one number token.
- It has two to four text tokens.



- It has no postcode.
- It looks like an address phrase, not a product sentence.

The backend first tries the full address. If the exact house point is missing, it removes the house number and tries the street-level address. The response marks whether the origin came from `address` or `address_street`.

## 25.22 Product and service concept handling

The program has local concept knowledge for broad product and service requests. The goal is not to maintain a giant table of every possible product. The goal is to map common product families to OSM categories and tags that can exist in the PBF.

Examples:

- A medicine-like query can map to pharmacies or chemists.
- A grocery-like query can map to supermarkets, convenience shops, or relevant retail categories.
- A car-brand query can use stored brand/name text and shop/service categories.
- A fast-food brand query can use POI name, brand, and restaurant/fast-food category.

The backend still validates against local data. If the local PBF has no matching POI, the result should be empty or weaker; the program should not create a fake shop.

## 25.23 Browser map behavior

The browser is a client for the local HTTP server.

- It requests only the current viewport instead of downloading all Europe objects.
- Orange markers show house/address points.
- Blue lines show streets.
- Green polygons show admin areas.
- Purple markers show forward-geocoder result POIs/admin/street/house candidates.
- Clicking a result pans or highlights that specific ranked result.
- The reverse-geocoder checkbox controls whether map clicks trigger reverse lookup.
- The Ollama button tries to start local Ollama service when it is not already reachable.

## 25.24 Viewport loading

Viewport loading protects the browser from receiving millions of records at once.

1. Leaflet reports the current map bounds.
2. The frontend calls viewport endpoints with min/max latitude and longitude.
3. The server uses spatial indexes to find objects intersecting the bounding box.
4. The server returns only a limited JSON slice for that view.
5. When the user pans or zooms, the browser requests a new slice.

This is separate from geocoder ranking. The search index can search the full loaded Europe snapshot, while map drawing remains viewport-limited.

## 25.25 Binary snapshot in detail

The binary snapshot stores the compact dataset and forward index so the user does not need to parse the PBF every time.

1. Write a magic header and version information.
2. Write selected language.
3. Write the string table.
4. Write records for houses, streets, admin areas, POIs, and links.
5. Write shared geometry arrays.
6. Write admin ring metadata, including outer and inner rings.
7. Write extraction statistics and timings.
8. Write the forward geocode index.

Old snapshots without the embedded forward index are rejected. This avoids silently rebuilding a huge forward index at server startup and avoids confusing runtime behavior.

## 25.26 Server startup after loading a snapshot

After `-load-binary`, the server still builds spatial grids. These grids are runtime structures used for fast viewport requests, reverse geocoding, and nearest-POI search. They are smaller and simpler than the full parser state, but they are still needed because they depend on how the HTTP server serves spatial requests.

The forward geocode index is already in the snapshot. That is why normal search does not need to rebuild the forward index at server start.

## 25.27 Progress bars, timings, and metrics

Progress bars measure byte position in the PBF where libosmium exposes enough information. ETA is shown only after enough progress history exists. At the very beginning it can be unknown because there is not enough speed history to predict the remaining time.

The final summary reports:

- scanned nodes, ways, and relations
- extracted houses, streets, POIs, and admin areas
- missing address fields
- admin links and parent links
- phase timings
- input throughput
- compact dataset size
- peak RSS memory

## 25.28 Memory reduction rules

The implementation keeps memory lower by removing parser-only state before expensive later phases.

- Strings are interned once, then the hash-map index is released before admin attribution.

- Relation way geometry uses flat coordinate storage and spans instead of many separate vectors.
- Needed-node found flags are packed more tightly.
- Forward posting lists are moved from temporary maps instead of copied.
- Vocabulary is built from sorted posting lists instead of a large temporary hash set.
- Geometry offsets use checked 32-bit casts so overflow becomes a clear error instead of silent corruption.

These changes reduce duplicate storage. They do not remove the real map objects that are required for the implemented features.

## 26 Commands

### 26.1 Build

```
cd /path/to/osm_project
cmake -S . -B /path/to/osm_project-build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build /path/to/osm_project-build
```

### 26.2 Parse Europe

```
cd /path/to/osm_project
/path/to/osm_project-build/osm_parser \
  --parse /path/to/data/europe-latest.osm.pbf \
  --low-memory \
  --connect-streets \
  --save-binary /path/to/data/europe-geocoder.bin \
  --pbf-threads 24
```

### 26.3 Load and serve

```
cd /path/to/osm_project
OSM_OLLAMA_MODEL=qwen2.5:3b \
OSM_OLLAMA_TIMEOUT_SECONDS=45 \
/path/to/osm_project-build/osm_parser \
  --load-binary /path/to/data/europe-geocoder.bin \
  --server 8080
```

### 26.4 Open browser

<http://localhost:8080>

## 27 Closing mental model

The project is best understood as four layers:

1. **Extraction layer:** read PBF, keep selected objects, skip irrelevant raw data.
2. **Geometry layer:** build street lines, admin polygons, holes, bboxes, and admin links.
3. **Index layer:** build text indexes, suffix index, fuzzy expansion support, and spatial grids.
4. **Application layer:** serve map tiles overlay data, reverse geocode clicks, forward geocode text, and optionally parse natural sentences through local Ollama.

Every visible result comes from the loaded PBF-derived dataset. The frontend, ranking system, and LLM mode are ways to ask the local dataset better questions; they are not external geocoding sources.